



APLIKACE AKRMAT – SPRÁVA HLAVNÍCH OBJEKTŮ A GENERÁTORY

Kryštof Jelínek

Bakalářská práce
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Studijní program: Informatika
Specializace: Softwarové inženýrství 2021
Vedoucí: Ing. Michal Valenta, Ph.D.
8. května 2026

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Jelínek** Jméno: **Kryštof** Osobní číslo: **515982**
Fakulta/ústav: **Fakulta informačních technologií**
Zadávající katedra/ústav: **Katedra softwarového inženýrství**
Studijní program: **Informatika**
Specializace: **Softwarové inženýrství**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Aplikace Akrmat - správa hlavních objektů a generátory

Název bakalářské práce anglicky:

Akrmat application - management of main objects and generators

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Michal Valenta, Ph.D. katedra softwarového inženýrství FIT

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **06.11.2025**

Termín odevzdání bakalářské práce: _____

podpis vedoucí(ho) ústavu/katedry

podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

_____ Datum převzetí zadání

Jelínek Kryštof
_____ Podpis studenta

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Jelínek** Jméno: **Kryštof** Osobní číslo: **515982**
Fakulta/ústav: **Fakulta informačních technologií**
Zadávající katedra/ústav: **Katedra softwarového inženýrství**
Studijní program: **Informatika**
Specializace: **Softwarové inženýrství**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Aplikace Akrmat - správa hlavních objektů a generátory

Název bakalářské práce anglicky:

Akrmat application - management of main objects and generators

Pokyny pro vypracování:

Práce navazuje na dva týmové projekty a jednu diplomovou práci, ve kterých došlo k přepisu aplikace Akrmat z PHP do Kotlin a Javascript a také ke změně datového úložiště.

Cílem této práce je revize a dokončení práce s již podporovanými entitami Předmět, CV, Program a Specializace, přidání podpory dalších objektů, které jsou zdrojem pro další formuláře akreditačních materiálů (AI, BI, CIV, CV, D, E, ...), revize a dokončení generátorů LaTeX, PDF a zbyde-li čas i dalších formátů.

Postupujte v těchto krocích:

1. Prostudujte požadavky na generování akreditačních materiálů - zejména požadavky na strukturu a obsah jednotlivých formulářů, ze kterých sestává akreditační spis.
2. Ověřte, zda již existující podpora správy základních objektů nového systému akrmat podporuje všechny požadavky na výsledné formuláře, případně doplňte chybějící položky.
3. Navrhněte strukturu entit, které budou spravovat data dalších (dosud nepodporovaných) formulářů, implementujte je v databázovém stroji PostgreSQL s využitím JSON a nad nimi navrhněte a implementujte backend a frontend pro správu těchto entit.
4. Ve stejném duchu zrevidujte a doplňte subsystém pro generování LaTeX zdrojových souborů a PDF výstupů.
5. Analyzujte, navrhněte a zbyde-li čas na zvoleném příkladu implementujte, jak by bylo možné systém rozšířit o generování dalších formátů základních entit (např HTML nebo JSON).
6. Všechny změny a nové části implementace a návrhu řádně zdokumentujte a otestujte.

Seznam doporučené literatury:

České vysoké učení technické v Praze

Fakulta informačních technologií

© 2026 Kryštof Jelínek. Všechna práva vyhrazena.

Tato práce vznikla jako školní dílo na Českém vysokém učení technickém v Praze, Fakultě informačních technologií. Práce je chráněna právními předpisy a mezinárodními úmluvami o právu autorském a právech souvisejících s právem autorským. K jejímu užití, s výjimkou bezúplatných zákonných licencí a nad rámec oprávnění uvedených v Prohlášení, je nezbytný souhlas autora.

Odkaz na tuto práci: Jelínek Kryštof. *Aplikace Akrmat – správa hlavních objektů a generátorů*. Bakalářská práce. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

Chtěl bych poděkovat především vedoucímu práce Ing. Michalovi Valentovi, Ph.D., za jeho odborné vedení, cenné rady a čas, který mi v průběhu zpracování práce věnoval. Dále patří mé velké poděkování mým rodičům a přítelkyni za jejich trpělivost, pochopení a podporu, kterou mi poskytovali nejen během psaní této práce, ale po celou dobu mého studia.

Prohlášení

Prohlašuji, že jsem předloženou práci vypracoval samostatně a že jsem uvedl veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací.

Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil. Stvrzuji, že jsem si vědom, že za obsah závěrečné práce plně zodpovídám.

Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona, ve znění pozdějších předpisů, zejména skutečnost, že České vysoké učení technické v Praze má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 citovaného zákona.

V Praze dne 8. května 2026

Abstrakt

Tato bakalářská práce se věnuje rozšíření a úpravě nové generace informačního systému Akrmát, který slouží k přípravě podkladů pro akreditace studijních programů na Fakultě informačních technologií ČVUT v Praze. Práce implementuje dosud nepodporované akreditační formuláře a strukturní prvky dokumentu, které v systému chyběly. V rámci práce je provedena analýza stávajícího stavu aplikace a navrženo řešení. Implementace zahrnuje rozšíření datové vrstvy, doménového modelu, uživatelského rozhraní a sázecí mikroslužby. Práce se dále zaměřuje na refaktORIZACI této mikroslužby s cílem odstranit technický dluh a vyřešit předešlé nedostatky v procesu kompilace a bezpečnosti zpracování. Součástí práce je také přidání nového exportu dat entity Předmět do formátu Markdown. Výsledkem je systém umožňující generování kompletních materiálů bez nutnosti dodatečného zpracování.

Klíčová slova Akrmát, akreditační materiály, refaktORIZACE, studijní program, generování dokumentů, mikroslužba, LaTeX, Kotlin, Spring Boot

Abstract

This bachelor thesis focuses on the expansion and modification of the new generation of the Akmat information system, which is used for the preparation of materials for the accreditation of study programs at the Faculty of Information Technology, CTU in Prague. The thesis implements previously unsupported accreditation forms and structural document elements that were missing in the system. The thesis includes an analysis of the application's current state and a proposed solution. The implementation involves the expansion of the data layer, the domain model, the user interface, and the typesetting microservice. Furthermore, the work focuses on refactoring this microservice to eliminate technical debt and resolve previous deficiencies in the compilation process and processing security. The thesis also includes the addition of a new data export for the "Course" entity into the Markdown format. The result is a system enabling the generation of complete materials without the need for additional processing.

Keywords Akmat, accreditation materials, refactoring, study program, document generation, microservice, LaTeX, Kotlin, Spring Boot

Obsah

1	Úvod	1
1.1	Motivace	2
1.2	Cíle práce	2
1.3	Struktura práce	3
2	Kontext a specifikace domény	4
2.1	Základní pojmy	4
2.2	Akreditační formuláře	5
2.3	Právní a legislativní rámec akreditace studijních programů . . .	7
3	Technologický kontext	9
3.1	Frontend	9
3.1.1	React	9
3.1.2	TypeScript	10
3.2	Backend	10
3.2.1	Kotlin	10
3.2.2	Spring a Spring Boot	11
3.2.3	OpenFeign	11
3.2.4	Hibernate	12
3.2.5	REST API	12
3.3	Databáze	13
3.3.1	Objektové databáze	13
3.3.2	Relační databáze	13
3.3.3	Flyway	14
3.4	LaTeX	14
3.5	Docker	15
4	Analýza současného stavu	16
4.1	Architektura aplikace	16
4.2	Frontend	17
4.2.1	Analýza současných nedostatků	18
4.3	Backend	18
4.3.1	Architektura	19
4.3.2	Analýza současných nedostatků	20
4.4	Databáze	20
4.5	LaTeX mikroslužba	21

4.5.1	Analýza současných nedostatků	22
4.6	Proces generování akreditačního spisu	23
5	Rozbor požadavků a návrh řešení	25
5.1	Specifikace požadavků	25
5.1.1	Korekce existujících výstupů	26
5.1.2	Rozšíření o nové formuláře	26
5.1.3	Další požadavky	27
5.2	Návrh řešení	28
5.2.1	Úpravy stávajících výstupů	28
5.2.2	Návrh nových přehledů z existujících dat	28
5.2.3	Návrhy správy nových formulářů	29
5.2.3.1	Koncept fixního formulářového rozhraní	29
5.2.3.2	Koncept přímé manipulace s obsahem	30
5.2.3.3	Diskuse a zdůvodnění výběru	30
5.2.4	Návrh refaktORIZACE mikroslužby	31
6	Implementace	32
6.1	Rozšíření datového úložiště	32
6.2	Rozvoj aplikační logiky	33
6.2.1	Rozšíření doménového modelu	33
6.2.2	Revize mapovacích komponent	36
6.2.3	Generování souhrnného seznamu vyučujících	36
6.2.4	Export do formátu Markdown	37
6.2.5	Aktualizace aplikačního rozhraní	39
6.3	RefaktORIZACE a rozšíření LaTeX mikroslužby	39
6.3.1	Strukturální změny aplikace	40
6.3.2	Izolace pracovních adresářů	40
6.3.3	Kompilační proces	40
6.4	Změny v uživatelském rozhraní	41
6.5	Testování a dokumentace	42
6.5.1	Dokumentace	42
6.5.2	Testování	42
7	Závěr	44
A	Zdrojové kódy	46
	Bibliografie	47
	Obsah příloh	50

Seznam obrázků

2.1	Vzor přílohy B-III ve formátu <code>.docx</code> převzaté z [3]	7
4.1	Ukázka uživatelského rozhraní	18
4.2	Zjednodušené logické schéma databáze převzaté z [23]	21
6.1	Ukázka ovládacích prvků nových formulářů	41
6.2	Ukázka ovládacích prvků pro nové exporty	42

Seznam tabulek

4.1	Přehled endpointů LaTeX mikroslužby	22
6.1	Přehled přidanych endpointů	39

Seznam výpisů kódu

1	Ukázka React komponenty	10
2	Příklad definice klienta pomocí OpenFeign	12
3	Migrační skripty pro rozšíření tabulek programme a course	33
4	Definice výčtového typu a ukázka využití	35
5	Šablona pro řádek seznamu vyučujících	37
6	Ukázka CourseMarkdownMapping objektu a jeho použití	38
7	Ukázka začátku Markdown šablony	39
8	Ukázka testu výčtového typu	43

Seznam zkratek

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CV	Curriculum Vitae (životopis)
ČVUT	České vysoké učení technické
DI	Dependency Injection
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IoC	Inversion of Control
Javadoc	Java Documentation
JPA	Java Persistence API
JSON	JavaScript Object Notation
JSX	JavaScript XML
JVM	Java Virtual Machine
JWT	JSON Web Token
KDoc	Kotlin Documentation
KOS	KOMponenta Studium - studijní informační systém ČVUT
MUI	Material UI
NAÚ	Národní akreditační úřad
ORM	Object Relational Mapping
PDF	Portable Document Format
PZ	Předmět profilujícího základu
REST	Representational State Transfer
RVH	Rada pro vnitřní hodnocení
SEO	Search Engine Optimization
SPA	Single Page Application
SQL	Structured Query Language
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
VŠ	Vysoká škola
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language
ZT	Základní teoretický předmět



Kapitola 1

Úvod

Příprava podkladů pro akreditaci studijních programů představuje pro Fakultu informačních technologií ČVUT v Praze rozsáhlý administrativní proces. Klíčovým nástrojem, který napomáhá zajištění tohoto procesu, je aplikace Akrmát. Ta slouží jako systém pro správu informací o studijních plánech, předmětech, vyučujících či jejich publikacích a umožňuje z těchto dat automatizovaně generovat finální dokumenty pomocí systému LaTeX.

Původní verze aplikace, vyvíjená v rámci bakalářských a diplomových prací v jazyce PHP, se v současné době blíží k hranicím své životnosti. Kvůli technologickému stárí je její údržba náročná a implementace nových požadavků fakulty prakticky nemožná. Z toho důvodu byly zahájeny práce na nové generaci systému, která využívá modernější technologický stack – konkrétně jazyk Kotlin pro backend a framework React pro frontend. Vzhledem k tomu, že se tato práce věnuje výhradně rozvoji této nastupující generace, bude v celém následujícím textu označení „systém Akrmát“ (případně „aplikace Akrmát“, „současné řešení“ apod.) referovat právě k tomuto novému systému, není-li výslovně uvedeno jinak.

Tato bakalářská práce navazuje na výsledky dřívějších vývojových cyklů. Ačkoliv byla kostra backendu nové aplikace již vytvořena jako praktická část diplomové práce Ing. Jana Štefánika, systém vykazoval funkční i technické nedostatky, které bránily jeho ostrému nasazení. Jednalo se především o problémy s integritou dat, chybějící aplikační logiku pro správu specifických formulářů a technické komplikace při generování výstupů.

Během stabilizace systému došlo k výměně datové vrstvy, kdy byla databáze MongoDB nahrazena systémem PostgreSQL. Tato změna umožnila efektivnější práci se strukturovanými daty v kombinaci s formátem JSON. Problematiku integrity těchto dat řešil ve své bakalářské práci Bc. Zdeněk Krajíček, který implementoval validaci pomocí JSON schémat a nasadil dedikovaný plugin pro jejich správu.

V současné době probíhá vývoj aplikace paralelně s dalšími pracemi. Zatímco tato práce se soustředí na implementaci nových formulářů a finalizaci

procesu kompilace, Sára Strnadová se ve své práci *Aplikace Akrmát – zajištění konzistence dat a rozšíření o podpůrné a provozní funkcionality* věnuje hlubší analýze integrity a konzistence dat napříč celým systémem. Správu prostředí a implementaci end-to-end testů pak pokrývá práce Matúše Walla s názvem *Aplikace Akrmát – DevOps pro vývojové, testovací a produkční prostředí, end2end testy*.

Zvolené téma navazuje na mou předchozí činnost v rámci týmového projektu, během kterého jsem implementoval samostatnou mikroslužbu zajišťující kompilaci zdrojových kódů systému LaTeX do výsledných dokumentů ve formátu PDF. Pokračování v tomto tématu mi umožňuje využít dříve nabytou znalost architektury systému k efektivnější realizaci úprav napříč všemi vrstvami aplikace a k odstranění nedostatků v dříve vytvořeném řešení. Možnost kriticky zhodnotit a vylepšit vlastní kód s odstupem času považuji za významný přínos pro kvalitu výsledné práce i pro můj osobní rozvoj.

1.1 Motivace

Jak bylo zmíněno v úvodu, původní verze aplikace Akrmát již nedostačuje dynamicky se měnícím požadavkům na strukturu a obsah akreditačních spisů. Problémem nového systému je, že zatím nepokrývá veškerou funkcionalitu nezbytnou k automatizaci celého procesu. Nedávné změny ve formátu specifických formulářů a úplná absence podpory jiných formulářů a strukturních prvků dokumentu nutí správce aplikace k pracným manuálním úpravám vygenerovaných výstupů. To nejen zvyšuje časovou náročnost přípravy podkladů, ale zároveň vnáší do celého procesu i riziko lidské chyby. Motivací práce je proto tyto nedostatky odstranit a modifikovat novou generaci systému do takové podoby, aby umožnila generování kompletní a korektní dokumentace bez nutnosti dodatečných ručních zásahů, čímž bude zajištěna praktická použitelnost systému v reálném provozu fakulty.

1.2 Cíle práce

Hlavním cílem práce je rozšíření nového systému Akrmát o dosud nepodporované akreditační formuláře a strukturní prvky dokumentu, jako jsou titulní strana či úvodní texty (pro zjednodušení a plynulost dalšího textu budou pojmy jako „nové formuláře“, „akreditační přílohy“ či obdobná označení zahrnovat i tyto strukturní prvky). V rámci implementace tohoto cíle je realizována úprava databázového schématu, vytvoření aplikační logiky pro správu nových dat a doplnění uživatelského rozhraní o ovládací prvky. Tyto nové formuláře jsou následně integrovány do existujícího subsystému zajišťujícího kompilaci LaTeX souborů. Souvisejícím cílem práce je revize aktuálního stavu entit *Předmět*, *Vyučující* a *Program* s cílem zajistit, aby generované výstupy plně odpovídaly současným požadavkům fakulty na akreditační dokumentaci. Aby se

tyto změny promítly do finálního PDF výstupu, zahrnuje práce také revizi subsystému pro kompilaci zdrojových souborů systému LaTeX, která zajistí správnou sazbu všech nových i upravených objektů. Celý proces je zakončen otestováním implementovaných funkcí a zdokumentováním řešení.

1.3 Struktura práce

Úvodní kapitola stručně představuje aplikaci Akrmát, její historii a účel. Po úvodní kapitole následuje vymezení domény akreditačního procesu, definice základních pojmů a popis požadavků na strukturu formulářů. Třetí kapitola představuje technologický kontext práce, konkrétně zvolený technologický stack zahrnující framework React, jazyk Kotlin, databázi PostgreSQL a sázecí systém LaTeX. Čtvrtá kapitola obsahuje analýzu současného stavu aplikace a identifikuje slabá místa v procesu generování dokumentů. Na základě této analýzy se pátá kapitola věnuje rozboru požadavků a návrhu řešení, včetně rozšíření datového modelu a úprav uživatelského rozhraní. Šestá kapitola detailně popisuje samotnou implementaci nových funkcí a refaktORIZACI LaTeX subsystému. Práci uzavírá sedmá kapitola, která shrnuje dosažené výsledky a navrhuje možnosti dalšího rozvoje systému.

Kontext a specifikace domény

V této kapitole je poskytnut ucelený pohled na problematiku akademických akreditací na ČVUT v Praze s popisem metodiky Národního akreditačního úřadu. Cílem textu je vybudovat teoretický a doménový základ skrze definici základní terminologie, struktury povinné dokumentace a legislativního rámce. Takto definovaný kontext slouží jako východisko pro pochopení motivace k vývoji systému Akrmat.

Akademická akreditace na ČVUT se řídí metodikou Národního akreditačního úřadu (NAÚ) a vnitřními předpisy univerzity, přičemž hlavní roli v procesu vnitřního hodnocení zastává Rada pro vnitřní hodnocení (RVH). Díky institucionální akreditaci disponuje ČVUT oprávněním v určených oblastech vzdělávání samostatně vytvářet a vnitřně schvalovat studijní programy [1]. V ostatních případech je akreditace udělována přímo prostřednictvím NAÚ. Technická náročnost procesu spočívá především v nutnosti transformovat velké množství hierarchicky provázaných dat, která zahrnují studijní plány, charakteristiky předmětů i personální údaje vyučujících, do podoby strukturovaných akreditačních formulářů. Obtížným úkolem při manuálním zpracování těchto podkladů je udržení konzistence dat, jakákoliv změna v parametrech studijního programu či personálním zabezpečení vyžaduje opravy v několika různých dokumentech. Účelem systému Akrmat je tento proces centralizovat a zajistit integritu dat tím, že se veškeré úpravy v databázi automaticky promítají do všech generovaných výstupů, čímž se eliminuje riziko nekonzistence údajů a zajišťuje se shoda s formálními požadavky popsány v [2].

2.1 Základní pojmy

Pro správné pochopení problematiky generování akreditačních materiálů je nezbytné definovat základní pojmy, se kterými systém Akrmat pracuje. Následující pojmy vycházejí z metodických pokynů Národního akreditačního úřadu [2].

- **Studijní program** je základní jednotka vysokoškolského vzdělávání, která směřuje k dosažení konkrétního stupně vzdělání (např. bakalářského). Dokument NAÚ jej definuje jako soubor studijních předmětů a pravidel pro jejich absolvování. Program je charakterizován typem, formou studia a profilem (akademický či profesní) [2]. V aplikaci Akrmat tvoří kořenový prvek, na který jsou navázány všechny ostatní entity.
- **Specializace** je vymezenou a obsahově ucelenější částí studijního programu, která jej blíže zaměřuje. Podle metodiky NAÚ [2] musí mít program se specializacemi společný základ (společnou část studijního plánu), přičemž specializace se od sebe liší specifickými bloky předmětů, které prohlubují znalosti v konkrétní oblasti.
- **Studijní předmět** je základní jednotka studijního plánu. Každý předmět v systému obsahuje například informace o svém rozsahu (přednášky a cvičení), počtu kreditů a anotaci. Předmět dále v rámci studijního programu spadá do jedné z kategorií: povinné, povinně volitelné a volitelné [2]. Pro účely akreditace je také podstatná vazba předmětu na jeho garanta a vyučující.
- **Garant** je role akademického pracovníka, který odpovídá za kvalitu a odbornou úroveň celého programu nebo předmětu. Metodika NAÚ klade na tuto osobu přísné nároky z hlediska její odborné a publikační činnosti (zejména v posledních pěti letech). [2]
- **Vyučující (Akademický pracovník)** je osoba, která se podílí na uskutečňování výuky předmětu. V zadání práce je tato entita označována jako CV, v textu je však upřednostněn termín vyučující pro jasné odlišení od akreditačního formuláře C-V (finanční zabezpečení programu). Systém o vyučujícím spravuje například informace o vzdělání, úvazcích, vedení prací či publikační činnosti. Tyto údaje jsou nutné pro doložení personálního zabezpečení programu. [2]
- **Akreditační formulář** je dokument s pevnou strukturou definovaný regulátorem (NAÚ nebo Rada pro vnitřní hodnocení). Tyto formuláře mají definovanou strukturu (označovanou písmeny A–G), kterou systém Akrmat automatizovaně naplňuje daty z databáze.

2.2 Akreditační formuláře

Proces schvalování studijních programů vyžaduje předložení strukturované dokumentace formou standardizovaných akreditačních příloh. V zájmu stručnosti a s ohledem na zaměření této práce jsou v následujícím textu popsány pouze relevantní části příloh A až E, přičemž jejich popis vychází z [3]. Tyto přílohy definují aspekty navrhovaného programu a tvoří předmět datového zpracování v systému Akrmat.

- **Příloha A – Základní informace o žádosti** (tvořená jediným formulářem **A-I**) obsahuje název vysoké školy a její součásti, na které má být studijní program uskutečňován. Uvádí se zde typ žádosti, tedy zda jde o udělení, prodloužení nebo rozšíření akreditace. Součástí jsou také odkazy na elektronickou podobu žádosti, relevantní vnitřní předpisy a studijní opory určené pro distanční či kombinovanou formu studia. Nezbytným údajem je zařazení programu do oboru vzdělání podle klasifikace ISCED-F 2013 včetně zdůvodnění tohoto návrhu. [3]
- **Příloha B – Charakteristika programu a studijní plány** představuje stěžejní část dokumentace popisující obsahové zaměření studia. Dělí se na několik podformulářů, jejichž specifikace je převzata z [3].
 - **B-I:** Obsahuje základní parametry jako název, typ, formu, profil, standardní dobu studia a vyučovací jazyk. Dále definuje cíle studia, profil absolventa (odborné znalosti a dovednosti) a jeho předpokládanou uplatnitelnost na trhu práce.
 - **B-IIa:** Definuje konkrétní studijní plány, rozsah výuky v hodinách, kreditové ohodnocení, způsob ověření výsledků učení a jmenný seznam vyučujících. U předmětů se rozlišuje, zda patří do profilujícího základu (PZ) nebo jsou základními teoretickými předměty (ZT).
 - **B-III:** Obsahuje detailní syllabus každého předmětu včetně jeho garanta, osnovy přednášek s hlavními tématy, metod výuky a seznamu povinné i doporučené literatury.
- **Příloha C – Zabezpečení studijního programu** se zaměřuje na personální, informační a materiální zdroje. Specifikace všech níže uvedených částí je převzata z [3].
 - **C-I:** Personální zabezpečení vyžaduje podrobné údaje o každém vyučujícím, jejich pracovních úvazcích na VŠ, dosaženém vzdělání, zkušenostech s vedením prací a publikační či tvůrčí činnosti za posledních 5 let.
 - **C-II:** Shromažďuje informace o související tvůrčí, vědecké nebo umělecké činnosti vysoké školy (např. řešené granty a projekty), která se vztahuje k danému programu.
 - **C-III až C-V:** Popisují studijní informační systém, přístup k odborným databázím, materiální vybavení v podobě učeben a laboratoří a finanční zabezpečení programu.
- **Příloha D – Záměr rozvoje** (tvořená jediným formulářem **D-I**) obsahuje odůvodnění potřeby studijního programu a jeho vztah k dalším programům stejného zaměření. Dále podrobně popisuje systém výuky v případě kombinované nebo distanční formy studia. [3]

- **Příloha E – Sebehodnotící zpráva** představuje reflexi silných a slabých stránek vysoké školy a programu v kontextu akreditačních standardů. Obsahuje analýzu institucionálního prostředí a vyhodnocení kvality výuky, tvůrčí činnosti i personálního a materiálního zajištění. Zpráva musí být fakticky podložena konkrétními odkazy na data z ostatních příloh žádosti. [4]

Akrmat je navržen tak, aby automatizoval vyplňování zejména příloh typu B a C, které pracují s velkým objemem strukturovaných dat z databáze. Další detaily a ukázky vygenerovaných materiálů jsou k dispozici v příloze této práce.

B-III – Charakteristika studijního předmětu			
Název studijního předmětu			
Typ předmětu		doporučený ročník / semestr	
Rozsah studijního předmětu		hod.	kreditů
Prerekvizity, korekvizity, ekvivalence			
Způsob ověření výsledků učení		Forma výuky	
Forma způsobu ověření výsledků učení a další požadavky na studenta			
Garant předmětu			
Zapojení garanta do výuky předmětu			
Vyučující			
Hlavní témata a výsledky učení			

■ **Obrázek 2.1** Vzor přílohy B-III ve formátu .docx převzaté z [3]

2.3 Právní a legislativní rámec akreditace studijních programů

Proces přípravy a následné akreditace studijních programů na vysokých školách v České republice je striktně vymezen platnou legislativou, která stanovuje standardy kvality, organizační strukturu i povinnou dokumentaci. Legislativní základ této regulace je zákon č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.

Požadavky na akreditaci, jak jsou interpretovány v metodických pokynech Národního akreditačního úřadu [2, 3, 4], vycházejí zejména z následujících zákonných ustanovení:

- **Zákon č. 111/1998 Sb., o vysokých školách:**
 - **§ 44 a § 44a:** Definují věcný obsah studijního programu, jeho typ, formu, cíle studia a zařazení do konkrétních oblastí vzdělávání.

- **§ 79:** Stanovuje náležitosti žádosti o akreditaci a definuje obsah pro tuto práci podstatných příloh A až E, které zahrnují základní informace, charakteristiku programu, personální a materiální zabezpečení, záměr rozvoje a sebehodnotící zprávu.
- **§ 48 a § 49:** Upravují zákonné podmínky pro přijímací řízení a využívání kreditního systému jako kvantitativního vyjádření studijní zátěže.
- **Podzákonné normy a prováděcí předpisy:**
 - **Nařízení vlády č. 274/2016 Sb., o standardech pro akreditace ve vysokém školství:** Určuje požadavky na kvalifikaci garantů, personální kapacity a tvůrčí činnost nezbytnou pro uskutečňování programu.
 - **Nařízení vlády č. 275/2016 Sb., o oblastech vzdělávání ve vysokém školství:** Stanovuje základní tematické okruhy a rámcové profily absolventů pro jednotlivé specializace.
- **Zákon č. 137/2016 Sb. (novela zákona o VŠ):**¹ Představuje podstatný legislativní obrat, který zrušil dosavadní systém studijních oborů a nahradil jej koncepcí studijních programů s možností vnitřních specializací.

Uvedené zákony a nařízení tvoří závazný rámec, podle kterého NAÚ posuzuje každou žádost o akreditaci. Přesné znění těchto norem je veřejně přístupné a bylo pro účely této práce ověřeno v aktuální podobě [5].

¹Plným názvem: „Zákon, kterým se mění zákon č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a některé další zákony“.

Technologický kontext

Tato kapitola poskytuje stručný přehled o technologickém stacku informačního systému Akmat. Text popisuje nástroje frontendové i backendové části, mechanismy správy dat, sázecí systém LaTeX a platformu Docker. Cílem kapitoly je poskytnout čtenáři vhodný technologický úvod pro pochopení celého postupu řešení a technických rozhodnutí v navazujících částech práce.

3.1 Frontend

Klientská část systému Akrmat zahrnuje technologie pro tvorbu uživatelského rozhraní a zajištění logiky na straně webového prohlížeče. Následující podkapitoly popisují knihovnu React a jazyk TypeScript.

3.1.1 React

Frontend aplikace Akrmat využívá knihovnu React, což je open-source JavaScriptová knihovna vyvíjená společností Meta pro tvorbu uživatelských rozhraní. Vznikla v roce 2011, kdy ji Jordan Walke navrhl pro potřeby sítě Facebook s cílem zefektivnit správu dynamického obsahu. Základem Reactu je komponentová architektura, která umožňuje skládat rozhraní ze znovupoužitelných bloků obsahujících vzhled i logiku. Knihovna využívá deklarativní přístup, kdy vývojář definuje výsledný stav rozhraní a React se postará o jeho vykreslení, k čemuž se standardně využívá syntaktické rozšíření JSX kombinující JavaScript s prvky podobnými HTML. [6]

```
1 import { useState } from 'react';
2
3 export default function Counter() {
4   const [count, setCount] = useState(0);
5
6   return (
7     <div className="counter-container">
8       <p>Current value: {count}</p>
9       <button onClick={() => setCount(count + 1)}>
10         Increment
11       </button>
12     </div>
13   );
14 }
```

■ **Výpis kódu 1** Ukázka React komponenty

3.1.2 TypeScript

Pro zajištění vyšší kvality kódu a typové bezpečnosti je uživatelské rozhraní vyvíjeno v jazyce TypeScript. Tento open-source nástroj od společnosti Microsoft je definován jako nadmnožina (*superset*) JavaScriptu, kterou doplňuje o statickou typovou kontrolu. To umožňuje detekovat chyby již v průběhu vývoje, nikoliv až za běhu aplikace, což významně usnadňuje vývoj a správu rozsáhlejších projektů. Zdrojový kód je následně pomocí kompilátoru přeložen do čistého JavaScriptu, což zajišťuje plnou kompatibilitu se všemi moderními webovými prohlížeči. [7]

3.2 Backend

Serverová část aplikace zajišťuje zpracování dat, business logiku a komunikaci s externími moduly. Tato sekce popisuje nástroje a technologie tvořící serverovou infrastrukturu aplikace Akrmát.

3.2.1 Kotlin

Serverová část aplikace Akrmát je implementována v jazyce Kotlin. Jedná se o moderní, staticky typovaný programovací jazyk, který byl vyvinut společností JetBrains s důrazem na bezpečnost, stručnost a uživatelskou přívětivost. Poskytuje plnou interoperabilitu s jazykem Java, ale je navržen tak, aby řešil některé její nedostatky (např. nulové reference, které vedou k chybám *NullPointerException*) pomocí vestavěné nulové bezpečnosti (*null-safety*). [8]

Ačkoliv Kotlin přináší moderní syntaktické prvky a funkcionální paradigmatata, je plně kompatibilní s ekosystémem Java Virtual Machine (JVM). Zdrojový kód je kompilován do tzv. bytekódu, což umožňuje jeho spuštění na standardním prostředí JVM a bezproblémové využití stávajících Java knihoven a frameworků [8]. Této vlastnosti aplikace Akrmát využívá například pro integraci s frameworkem Spring Boot, který je detailněji popsán v následující podkapitole.

3.2.2 Spring a Spring Boot

Spring je open-source framework určený pro vývoj serverových aplikací, zejména v jazyce Java. Hlavní předností tohoto frameworku je, že výrazně redukuje objem redundantního kódu (tzv. boilerplate code) nezbytného pro zajištění základní infrastruktury serverové aplikace, jako je například obsluha databáze nebo vystavování API rozhraní. Hlavní principy, které Spring využívá k dosažení těchto výhod, jsou Inversion of Control (IoC) a Dependency Injection (DI). IoC přenáší zodpovědnost za řízení toku aplikace na framework a DI zajišťuje automatické propojování komponent, díky čemuž se programátor může soustředit pouze na samotnou aplikační logiku. [9]

Nevýhodou Springu je však nutnost rozsáhlé manuální konfigurace, kterou je nutné pro zprovoznění aplikace vykonat. Zde přichází na řadu Spring Boot, což je nadstavba Springu, která má tuto konfiguraci v sobě automaticky integrovanou. Spring Boot využívá princip konvence, což znamená, že manuální zásah do nastavení je nutný pouze v případě, kdy se požadované chování aplikace neshoduje s běžnou konvencí. [10]

3.2.3 OpenFeign

Problém propojení backendu aplikace Akrmát se samostatnou mikroslužbou určenou pro kompilaci dokumentů do formátu PDF je řešen pomocí nástroje OpenFeign. Jedná se o deklarativní REST klient, který zjednodušuje psaní webových služeb v prostředí Springu. Namísto manuálního sestavování HTTP požadavků umožňuje OpenFeign definovat rozhraní (*interface*) pouze pomocí Java anotací. Framework pak na pozadí automaticky vygeneruje implementaci tohoto rozhraní, která zajišťuje veškerou komunikaci se vzdáleným serverem. Vývojář tak se službou dále pracuje jako s běžnou lokální komponentou. [11]

```
1 @FeignClient(name="latex-service", url="${akrmat.latex_service.url}")
2 interface LatexClient {
3     @GetMapping("/teachers/{id}")
4     fun getTeacherPdf(@PathVariable id: String): ByteArray
5
6     @GetMapping("/courses/{id}")
7     fun getCoursePdf(@PathVariable id: String): ByteArray
8
9     @GetMapping("/programmes/{id}/withLogs")
10    fun getProgrammePdfWithLogs(@PathVariable id: String): ByteArray
11 }
```

■ **Výpis kódu 2** Příklad definice klienta pomocí OpenFeign

3.2.4 Hibernate

Pro práci s datovou vrstvou používá backend aplikace framework Hibernate. Jedná se o velmi populární nástroj pro objektově-relační mapování (ORM), což je technika, která umožňuje propojovat objekty v programovacích jazycích s jejich záznamy v databázi. Hibernate programátora zbavuje nutnosti psát SQL dotazy manuálně, místo toho lze Java objekty propojit s databázovými tabulkami pomocí anotací. Framework pak automaticky vygeneruje potřebné SQL dotazy a sám spravuje životní cyklus objektů [12]. V rámci ekosystému Spring Boot funguje Hibernate jako výchozí implementace specifikace JPA (*Jakarta Persistence API*) [13]. Poskytuje vrstvu abstrakce nad nízkoúrovňovou komunikací s databází, čímž snižuje náchylnost k chybám a zvyšuje čitelnost kódu.

3.2.5 REST API

REST (*Representational State Transfer*) je architektonický styl pro návrh webových rozhraní, který v roce 2000 definoval Roy Fielding. Celý koncept je založen na správě dat jakožto zdrojů (*resources*), které jsou jednoznačně identifikovány pomocí URI (*Uniform Resource Identifier*). V prostředí webových služeb pak tyto identifikátory typicky nabývají podoby konkrétních URL adres. Mezi nejdůležitější vlastnosti patří oddělení klienta od serveru a bezstavovost – každý požadavek musí být soběstačný a obsahovat veškeré informace nutné k jeho vyřízení, protože server mezi jednotlivými dotazy neuchovává žádná data o stavu klienta. Tento přístup umožňuje škálovatelnost celé aplikace. [14]

Dalším důležitým aspektem je oddělení zdroje od jeho reprezentace skrze tzv. *media types*. To znamená, že jeden konkrétní zdroj může mít více podob v závislosti na potřebách klienta. V kontextu aplikace Akrmát představují tyto zdroje entity jako *Předmět* nebo *Studijní program*. Zatímco pro potřeby

webového rozhraní jsou tato data typicky přenášena ve formátu JSON, systém může stejný zdroj poskytnout i v jiných formátech – například jako PDF dokument pro účely akreditace nebo ZIP archiv pro manuální úpravu podkladů zmíněného dokumentu. K manipulaci s těmito reprezentacemi se běžně využívají standardizované metody protokolu HTTP (zejména GET, POST, PUT a DELETE), které určují konkrétní prováděné operace. [14]

3.3 Databáze

Tato sekce se věnuje popisu databázových modelů a jejich vlastností. Dále je představen nástroj pro správu migrací databázového schématu.

3.3.1 Objektové databáze

Objektově orientované databáze ukládají data ve formě objektů, čímž přímo kopírují struktury používané v objektově orientovaném programování. Na rozdíl od relačních databází pracují s daty jako s instancemi tříd, které v sobě spojují data i související chování. Tento přístup programátorovi umožňuje ukládat složité datové struktury bez nutnosti komplikovaného převodu na řádky a sloupce, jako je tomu v případě relačních databází. To vede ke snazšímu vývoji a často i k vyššímu výkonu při zpracování komplexních datových struktur. [15]

Dříve byla v aplikaci Akrmát pro ukládání dat využívána databáze MongoDB, která jakožto dokumentový systém sdílí s objektovými databázemi vlastnosti, jako flexibilní schéma či přímé ukládání objektů bez nutnosti pevných relací. V průběhu vývoje však byla nahrazena relační databází PostgreSQL. K tomuto přechodu došlo především kvůli požadavku na striktnější kontrolu nad strukturou ukládaných dat, pro kterou je relační model vhodnější.

3.3.2 Relační databáze

Relační databáze představují v současnosti nejpoužívanější model pro ukládání dat v informačních systémech. V rámci tohoto modelu jsou veškerá data organizována do tabulek, přičemž do této podoby jsou převedeny jak samotná data, tak i vztahy mezi nimi. Každá tabulka se skládá z pevně definovaných sloupců (atributů) s unikátními názvy a z jednotlivých řádků, které reprezentují konkrétní datové záznamy. Pro definici struktury tabulek a následnou manipulaci s daty se stal celosvětovým standardem deklarativní jazyk SQL (*Structured Query Language*). Relační model klade velký důraz na udržování datové integrity, čehož dosahuje pomocí integritních omezení, jako jsou primární a cizí klíče. Tyto mechanismy umožňují definovat pevné vazby mezi tabulkami a zajišťují, že databáze zůstane v konzistentním stavu i při souběžném přístupu více uživatelů nebo v případě systémových selhání. [16]

3.3.3 Flyway

Pro správu změn v databázovém schématu využívá backend aplikace Akrmat nástroj Flyway. Jedná se o řešení pro migraci databází, které umožňuje verzovat strukturu databáze, čímž zajišťuje podobnou kontrolu nad vývojem schématu, jakou poskytuje systém Git pro správu zdrojového kódu. Veškeré úpravy schématu jsou definovány pomocí migračních SQL skriptů, které jsou součástí projektu.

Nástroj je integrován do procesu spouštění aplikace, během něhož identifikuje a provede změny. K tomu Flyway využívá tabulku *flyway_schema_history*, ve které uchovává záznamy o již aplikovaných skriptech. Pokud jsou při startu aplikace detekovány nové migrační soubory, dojde k jejich automatickému spuštění v předem definovaném pořadí. Tento přístup eliminuje potřebu manuálních zásahů do databáze na produkčním prostředí a zajišťuje konzistenci databázového schématu napříč všemi instancemi aplikace. [17]

3.4 LaTeX

LaTeX je systém určený pro přípravu dokumentů. Na rozdíl od běžných textových procesorů uživatel nepíše text přímo do jeho finální podoby, ale vytváří zdrojový soubor pomocí textového editoru. Tento soubor obsahuje kromě samotného textu také speciální notace, nazývané řídicí sekvence (*Control Sequences*), které popisují, jak má výsledný dokument vypadat – například určení tučného písma nebo definice nových kapitol. Za tvůrce systému LaTeX je považován Leslie Lamport, autor referenční příručky k tomuto systému. LaTeX je v podstatě sada maker postavená na starším sázecím systému TeX. [18]

Užívání LaTeXu je zcela odlišné od práce v běžných textových procesorech, jako je například Microsoft Word. Zatímco tyto procesory fungují na principu „co vidíš, to dostaneš“ (what you see is what you get), často postrádají funkce potřebné pro pokročilou automatizaci. LaTeX naopak vyniká v automatickém vytváření sekcí, poznámek pod čarou, obsahu či křížových odkazů. Flexibilitu systému a možnosti vytváření vlastního vzhledu dokumentu zajišťuje kromě řídicích sekvencí možnost definovat další styly v souborech s příponou .sty, které obsahují definice pro formátování výstupu [18]. Právě možnost automatizace a oddělení obsahu od jeho vizuální reprezentace jsou hlavními důvody, proč systém Akrmat využívá LaTeX k produkci formulářů. Systém přebírá data a vkládá je do předpřipravených šablon, které jsou následně zkompileovány do výsledného formátu PDF. Konkrétní technické řešení a způsob integrace LaTeXu do architektury aplikace budou detailně popsány v následujících kapitolách práce.

3.5 Docker

Docker představuje platformu určenou pro vytváření, nasazování a spouštění aplikací pomocí samostatných jednotek nazývaných kontejnery. Celý tento proces, označovaný jako kontejnerizace, umožňuje zabalit aplikaci se všemi jejími knihovnami a konfiguračními soubory do jednoho celku. Na rozdíl od virtuálních strojů, které vyžadují vlastní operační systém, je Docker tzv. „lightweight“, protože využívá přímo kernel hostitelského systému [19]. Mezi hlavní výhody kontejnerizace patří:

- **Portable (přenositelnost):** Zajišťuje, že aplikace poběží identicky v jakémkoliv prostředí.
- **Self-sufficient a encapsulated:** Kontejnery jsou izolované, zapouzdřené a obsahují vše potřebné pro svůj běh.
- **Scalable (škálovatelnost):** Architektura umožňuje snadné navyšování počtu běžících instancí aplikace.

Základním stavebním kamenem je takzvaný „image“ (obraz), což je statická šablona obsahující binární kód aplikace a veškeré závislosti. Samotný kontejner je pak aktivní, běžící instancí tohoto obrazu [19]. Pro systém Akrmat je tato technologie klíčová, protože zajišťuje stabilní a izolované prostředí pro všechny jeho části.

Analýza současného stavu

Tato kapitola se věnuje analýze aktuálního stavu systému Akmat a popisu procesu tvorby akreditačních materiálů. Hlavním cílem je identifikovat konkrétní technické a funkční nedostatky, které představují zdůvodnění pro realizaci úprav popsanych dále v této práci.

4.1 Architektura aplikace

Současná architektura systému Akrmat je koncipována jako distribuovaná aplikace sestávající ze tří volně vázaných komponent, které společně zajišťují celý proces od správy dat až po generování výsledných dokumentů. Prezentační vrstvu tvoří klientská část realizovaná jako aplikace v knihovně React, která zprostředkovává uživatelské rozhraní a komunikuje s ostatními částmi aplikace skrze REST rozhraní. Jádrem systému představuje serverová část implementovaná v jazyce Kotlin využívající framework Spring Boot. Tato komponenta zodpovídá za řízení doménové logiky a zajišťuje ukládání a načítání dat prostřednictvím relační databáze PostgreSQL.

Specifickou úlohu v rámci systému plní autonomní LaTeX subsystém napsaný v jazyce Java. Tato mikroslužba slouží jako dedikovaný nástroj pro kompilaci zdrojových kódů šablon do finální podoby PDF dokumentů prostřednictvím systému LaTeX. Zatímco uživatelské rozhraní interaguje přímo s backendem, komunikace mezi backendem a LaTeX mikroslužbou je realizována pomocí deklarativního REST klienta OpenFeign. Tento přístup umožňuje volání endpointů sázecí mikroslužby, jako by se jednalo o lokální součást hlavní aplikace, čímž je zachována přehlednost a rozšiřitelnost celého řešení.

Všechny komponenty systému jsou samostatně verzovány v rámci fakultního systému GitLab. Pro zajištění konzistentních závislostí a snadné přenositelnosti je každá část aplikace kontejnerizována pomocí Dockeru. Využití kontejnerizace je vhodné zejména pro LaTeX subsystém, u kterého izoluje rozsáhlé systémové závislosti nutné pro jeho správný chod. Celý proces sestavení a nasazení je navíc automatizován skrze CI/CD (*Continuous Integration /*

Continuous Deployment) pipeline, které při každé změně v repozitářích zajišťují automatické sestavení, otestování a následné nasazení částí aplikace do jejich cílového prostředí.

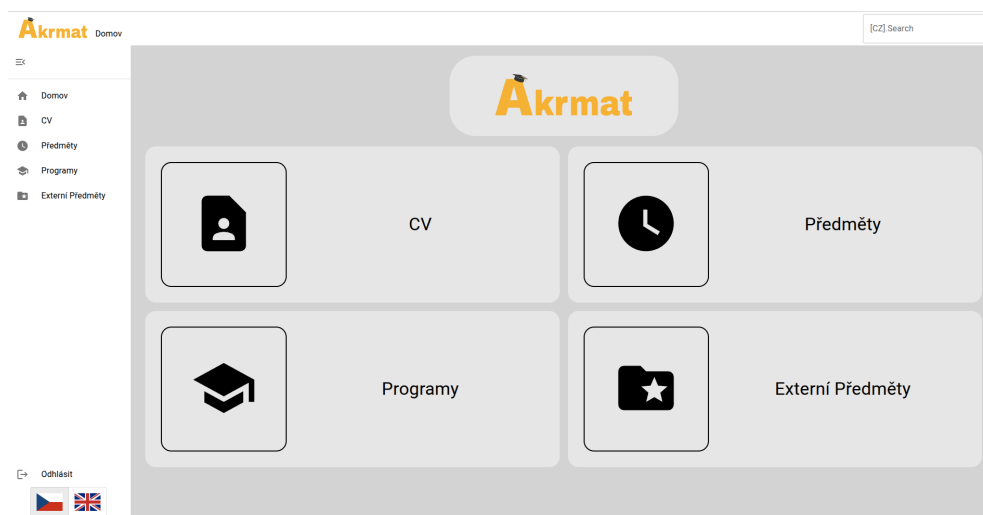
4.2 Frontend

Uživatelské rozhraní systému Akrmat je interaktivní webová aplikace, která slouží ke správě a vizualizaci datových entit. Rozhraní je logicky členěno do několika sekcí podle typu spravovaných objektů, jako jsou vyučující, studijní předměty či studijní programy. Přístup k jednotlivým funkcím je řízen na základě uživatelských rolí, které rozdělují uživatele na čtenáře s právem prohlížení a administrátory s možností data upravovat a ukládat změny zpět do systému. Aplikace je dále navržena jako multijazyčná a poskytuje lokalizaci do českého i anglického jazyka.

Z hlediska vnitřní architektury se jedná o jednostránkovou webovou aplikaci (*Single Page Application*). Tento přístup znamená, že veškerá interaktivita probíhá v rámci jednoho HTML dokumentu, který se po úvodním stažení dynamicky překresluje na základě uživatelských akcí a dat přijatých ze serveru. Tento model odstraňuje potřebu opětovného načítání celé stránky při přechodu mezi sekcemi, což umožňuje plynulejší práci se systémem.

Určitou nevýhodou architektury SPA bývá náročnější optimalizace pro vyhledávače (*Search Engine Optimization*, zkráceně SEO), jelikož dynamicky generovaný obsah může být pro roboty běžných vyhledávačů hůře indexovatelný [20]. V případě systému Akrmat však tento fakt nepředstavuje žádné omezení. Jedná se o interní aplikaci určenou výhradně pro potřeby Fakulty informačních technologií ČVUT, kde není vyžadována veřejná dohledatelnost ani indexace obsahu externími vyhledávači.

Pro vývoj byla využita knihovna React v kombinaci s typovým systémem TypeScript. Vizualní podoba rozhraní je definována komponentami knihovny *Material UI* (MUI), která poskytuje jednotný vzhled ovládacích prvků a responzivitu aplikace na různých typech zařízení [21].



Obrázek 4.1 Ukázka uživatelského rozhraní

4.2.1 Analýza současných nedostatků

Při rozboru aktuální podoby uživatelského rozhraní bylo identifikováno několik funkčních nedostatků, které tvoří překážku pro nasazení systému do ostrého provozu. Zásadním problémem je naprostá absence podpory pro správu a editaci specifických akreditačních příloh. V současném rozhraní zcela chybí moduly pro formuláře typu A-I, B-I, C-II až C-V, D-I nebo E. Tento nedostatek je dále umocněn fixní strukturou generovaného výstupu, která neumožňuje vytvořit titulní stranu dokumentu ani do něj vložit úvodní text. Správce aplikace tak nemá žádnou možnost tyto sekce vytvářet, ukládat či měnit jejich obsah pro konečný výstup akreditačního spisu.

Dalším zjištěným nedostatkem je nevyužitá funkcionality backendu v oblasti exportu zdrojových kódů. Přestože serverová část aplikace již obsahuje logiku pro generování archivů ZIP (obsahujících kompletní LaTeX podklady pro účely ladění a manuálních korekcí), frontend tuto funkci uživateli nijak nezprostředkovává. Uživatel je tak odkázán pouze na přímé generování PDF souborů. V případě chyby v sazbě nebo nutnosti individuálního zásahu do dokumentu je takový proces bez přístupu ke zdrojovým kódům neflexibilní a pro správce aplikace nepoužitelný.

4.3 Backend

Backendová část aplikace Akrmát zajišťuje veškerou business logiku, správu dat a poskytuje rozhraní ve formě REST API. To je navrženo tak, aby poskytovalo data primárně ve formátu JSON pro potřeby frontendu. U vybraných entit, jako jsou *Studijní program*, *Vyučující* nebo *Předmět*, však systém umožňuje získat jejich data i ve formátu HTML nebo `.tex`. Skrze rozhraní

FeignClient pak backend nepřímo zprostředkovává tyto entity i ve formátu PDF nebo kompletního ZIP archivu.

Aplikace je vyvíjena v programovacím jazyce Kotlin a využívá framework Spring Boot, přičemž volba Kotlinu přináší výhody statického typování a vestavěné bezpečnosti proti chybám typu *NullPointerException*. Pro správu závislostí a sestavení celého projektu slouží nástroj Maven. Mechanismus autentizace a autorizace v systému staví na bezstavovém ověřování pomocí JWT (*JSON Web Token*), kdy prvotní přihlášení probíhá přes fakultní OAuth, načež jsou z Usermap API načteny role uživatele a následně vygenerován token uložený v cookie. Samotný přístup k jednotlivým funkcím je pak řízen rolemi *READER*, *EDITOR* a *ADMIN*. K ověření funkčnosti aplikace slouží jednotkové testy v JUnit 5 a integrační testy využívající technologii Testcontainers. Tato technologie v původním řešení spouštěla instanci databáze MongoDB [22], která však byla v rámci týmových projektů nahrazena systémem PostgreSQL. Veškeré výše zmíněné technologie a mechanismy (které nejsou součástí kapitoly 3) jsou zde však zmíněny pouze okrajově v zájmu zachování plynulosti a směru textu, neboť jejich hlubší technický rozbor není pro pochopení následné implementace a cílů této práce nezbytný.

4.3.1 Architektura

Vnitřní struktura backendu je založena na principech hexagonální architektury, někdy také nazývané *Ports and Adapters*. Cílem tohoto přístupu je oddělení doménové logiky od technických detailů a vlivů externích systémů. Projekt je pro zajištění této separace rozdělen do dvou hlavních balíčků:

- **adapter:** Obsahuje veškerou logiku určenou pro komunikaci s okolím. Patří sem sub-balíčky *api* (REST endpointy pro komunikaci s frontendem), *persistence* (databázová vrstva) a *fit*, který sdružuje klienty pro externí fakultní systémy jako KOS či Usermap nebo například i pro mikroslužbu zajišťující kompilaci LaTeXu.
- **domain:** Představuje samotné jádro systému a obsahuje pouze doménové datové entity a business logiku. Ta je v kódu realizována pomocí tříd s příponou *Facade*, které orchestrují průběh servisních operací, a servisních tříd s příponou *Service*, které implementují konkrétní doménové operace.

Nezbytným prvkem pro udržení nezávislosti domény je mapovací vrstva. Každý adaptér pracuje s vlastním datovým modelem, který je následně transformován na doménové objekty pomocí *extension functions*. Tento mechanismus zajišťuje, že vnitřní logika aplikace zůstává konzistentní a není přímo ovlivněna změnami ve formátech dat externích systémů [22].

4.3.2 Analýza současných nedostatků

Obdobně jako na straně frontendu postrádá i backendová část systému Akrmát podporu pro správu a generování specifických akreditačních příloh typu A-I, B-I, C-II až C-V, D-I či E. Tyto dokumenty nejsou v aktuální verzi implementovány na žádné úrovni systému – od definice doménových entit přes aplikační logiku až po perzistentní vrstvu. V systému rovněž zcela chybí podpora pro strukturní části akreditačního spisu, jako je titulní strana, úvodní texty nebo samostatný souhrnný seznam vyučujících.

Kromě úplné absence zmíněných příloh akreditačního spisu vykazují i již existující entity (*Vyučující*, *Předmět* a *Program*), respektive jejich výsledné formuláře, rozdíly oproti finálním požadavkům na podobu dokumentace. Problém spočívá především v jejich neúplnosti a v chybějící nebo nesprávné logice pro mapování údajů z doménových entit do LaTeX šablon. Výsledkem jsou automaticky generované podklady, které postrádají některé informace vyžadované metodikou NAÚ, což v konečném důsledku znehodnocuje proces automatizace, a uvedení dokumentů do korektní podoby by vyžadovalo rozsáhlé manuální úpravy.

Dalším zjištěním, ke kterému došlo během analýzy současného řešení pro účely této práce, je technický nedostatek v servisní vrstvě aplikace, kde dochází ke vzniku duplicitních vazeb v databázi. Tato chyba představuje riziko pro integritu generovaných materiálů, nicméně v rámci této práce nebude její příčina ani oprava¹ detailně rozebírána. Její řešení je totiž součástí souběžně vznikající bakalářské práce Sárý Strnadové, která se věnuje hloubkové analýze konzistence dat napříč všemi moduly aplikace.

4.4 Databáze

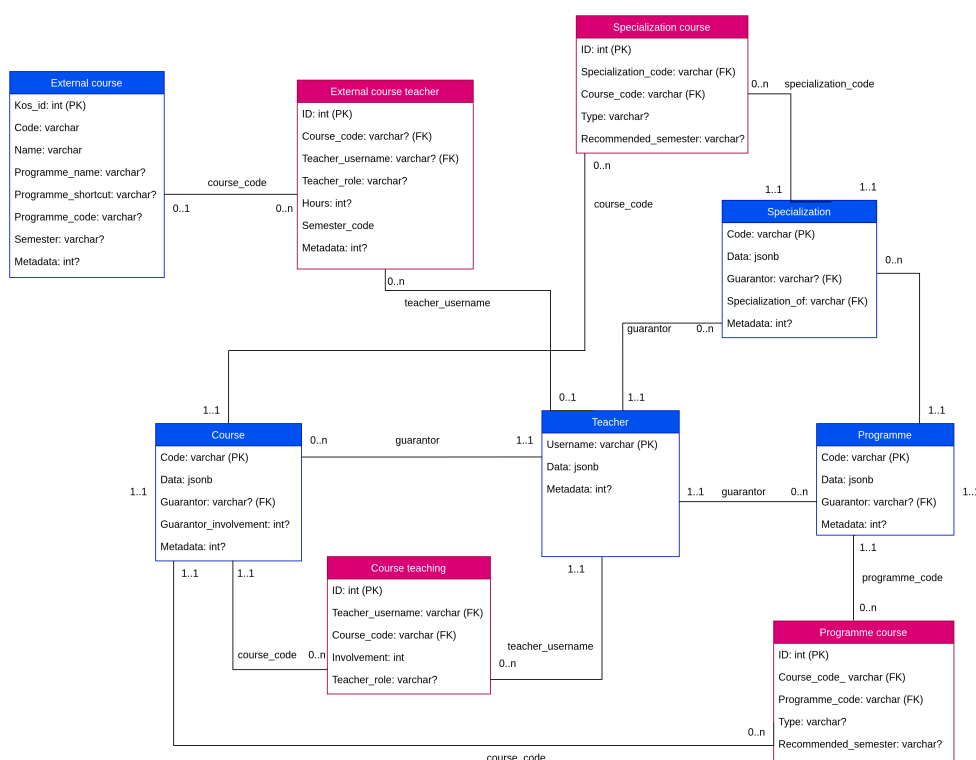
Pro ukládání a správu dat využívá aplikace Akrmát relační databázový systém PostgreSQL, na který byla zmigrována z původní dokumentové databáze MongoDB. Pro potřeby aplikace byl použit hybridní model kombinující prvky těchto dvou přístupů. Klíčové entity — **teacher**, **course**, **programme** a **specialization** — jsou reprezentovány tabulkami s fixními sloupci pro identifikátory a vazby, zatímco jejich ostatní data jsou uložena ve sloupci typu JSONB.

Zajištění integrity dat uvnitř JSON dokumentů řešil ve své bakalářské práci Bc. Zdeněk Krajíček [23]. Pro validaci struktury využil rozšíření **pg_jsonschema**. To umožňuje definovat JSON schémata přímo v databázi a vynucovat jejich dodržování pomocí integritních omezení **CHECK**. Tento přístup zaručuje, že do databáze nelze uložit objekt s chybnou strukturou, což je důležité pro korektní

¹Opravou je v tomto kontextu myšlen zásah do servisní logiky aplikace. Dopad chyby bude v rámci generování materiálů ošetřen využitím datové struktury tak, aby byl výstup korektní i v případě duplicitních dat.

generování výstupů. Správa databázového schématu a veškeré migrace jsou verzovány nástrojem Flyway, který zajišťuje konzistenci napříč všemi vývojovými prostředími.

Obdobně jako zbylé části aplikace i datové úložiště postrádá podporu pro ukládání dat specifických akreditačních příloh. V aktuálním návrhu tabulek ani v definovaných JSON schématech nejsou tyto dokumenty nijak zohledněny, což neumožňuje jejich správu a následné využití při generování kompletního akreditačního spisu.



Obrázek 4.2 Zjednodušené logické schéma databáze převzaté z [23]

4.5 LaTeX mikroslužba

Proces generování výsledných dokumentů v systému Akrmát zajišťuje samostatná mikroslužba napsaná v jazyce Java postavená na frameworku Spring Boot. Jejím účelem je kompilace zdrojových kódů LaTeX do formátu PDF. Služba je navržena jako samostatná aplikace, což backend zbavuje nutnosti spravovat systémové závislosti potřebné ke kompilaci LaTeXu.

Mikroslužba poskytuje rozhraní pro generování PDF dokumentů vyučujícího a předmětu, které má z hlediska celého procesu spíše podpůrnou funkci a slouží k individuální kontrole datových entit. Pro přípravu akreditačního spisu je určen endpoint pro studijní program, který v současném stavu generuje

PDF dokument obsahující pouze formuláře typu B-IIa, B-III a C-I. Ke každému z těchto endpointů existuje varianta poskytující výstup ve formátu ZIP (označená příponou `/withLogs`), která slouží k ladění výsledků. Přehled všech dostupných endpointů uvádí tabulka 4.1.

■ **Tabulka 4.1** Přehled endpointů LaTeX mikroslužby

Metoda	URL	Výstup
GET	<code>/courses/{id}</code>	PDF sylabus předmětu
GET	<code>/teachers/{id}</code>	PDF životopis vyučujícího
GET	<code>/programmes/{id}</code>	PDF studijního programu
GET	<code>/courses/{id}/withLogs</code>	ZIP archiv předmětu
GET	<code>/teachers/{id}/withLogs</code>	ZIP archiv vyučujícího
GET	<code>/programmes/{id}/withLogs</code>	ZIP archiv programu

Zmiňovaný formát ZIP obsahuje veškeré podklady nutné k lokální kompilaci, včetně zdrojových souborů, stylového souboru a výsledného PDF. Součástí archivu jsou také logovací a pomocné soubory přímo z procesu kompilace mikroslužby, konkrétně soubory s příponami `.aux` a `.log`. Vizuální podobu všech dokumentů určuje stylový soubor s příponou `.sty`, který funguje jako balíček maker pro definici typografie, rozvržení stran, velikosti okrajů a specifických příkazů pro sazbu. Aplikace pro tyto účely aktuálně využívá soubor s názvem `FITAK-24.sty`.

4.5.1 Analýza současných nedostatků

I když mikroslužba plní svou základní funkci, její implementace má několik slabých míst, která komplikují další vývoj. Tyto nedostatky se týkají jak samotné struktury aplikace, tak i kvality generovaných výstupů. Na první pohled patrným problémem je nedodržení principu *Separation of concerns* [24].

Téměř veškerá aplikační logika je soustředěna v jediné třídě `ApiController`. Výjimku tvoří pouze komponenta `ShellCommandExecutor`, která zajišťuje spouštění systémových příkazů. Kód služby není řádně rozdělen do funkcí nebo metod a kvůli chybějící abstrakci dochází v kódu k opakování podobných sekvencí, což zvyšuje redundanci a ztěžuje údržbu. Dalším zjištěním je úplná absence jednotkových a integračních testů, což neumožňuje ověřování funkčnosti systému při zavádění změn.

Z hlediska formátování výstupů je dalším nedostatkem využití stylového souboru `FITAK-24.sty`. Ten je v současné době zastaralý, neboť již existuje aktualizovaná verze `FITAK-25.sty`, která obsahuje úpravy formátu a drobné změny odpovídající současným požadavkům na podobu akreditačních materiálů.

Specifický technický problém nastává při kompilaci `.tex` souborů, které obsahují křížové odkazy (například na kódy předmětů). Protože systém LaTeX

potřebuje k řádnému provázání referencí nejdříve v průběhu první kompilace vygenerovat pomocné soubory a data z nich načíst až v dalším průchodu [18], výsledek kompilace v aktuálním řešení nemá tedy vždy tyto odkazy správně navázané. Mikroslužba v současnosti neprovádí potřebné opakované průchody kompilace, které by tento problém řešily.

Dalším nedostatkem je riziko vzniku takzvané *race condition* při pokusu o kompilaci více uživateli souběžně. Vzhledem k tomu, že aplikace využívá jeden společný pracovní adresář, mohou si uživatelé navzájem přepisovat obsah dočasných souborů, což může vést k nedefinovaným výsledkům. Přestože se nepředpokládá, že aplikaci bude používat velké množství uživatelů současně a přístup bude omezen na prověřené správce, představuje tento stav architektonickou vadu, kterou je nutné zmínit.

V poslední řadě v návaznosti na chybějící datovou podporu v backendové části systému postrádá i tato mikroslužba schopnost do výsledného dokumentu zahrnout zbývající akreditační přílohy typu A-I, B-I, C-II až C-V, D-I nebo E.

4.6 Proces generování akreditačního spisu

Aktuální proces generování akreditační dokumentace je realizován jako distribuovaná operace mezi frontendem, backendem a sázecí mikroslužbou. Celý proces začíná v uživatelském rozhraní, kde správce vyvolá požadavek na stažení konkrétní entity (*Vyučující*, *Předmět*, *Program*) ve formátu PDF nebo ZIP. Tento požadavek je z frontendu odeslán na backend, který jej prostřednictvím nástroje OpenFeign deleguje na LaTeX subsystém.

Samotná tvorba obsahu dokumentů probíhá na straně backendu. Ten využívá statické šablony uložené v adresáři `src/main/resources/form`. Tyto šablony obsahují volání maker definovaných v balíčku `FITAK-24.sty`, přičemž v těchto šablonách jsou předdefinované konstanty reprezentující textové zástupce (*placeholders*). Třídy typu `TexMapping` v aplikační logice backendu tyto zástupce nahrazují reálnými daty z databáze a tím generují výsledný zdrojový kód systému LaTeX². Tato data jsou ještě před nahrazením sanitována, aby došlo k ošetření (tzv. escapování) vyhrazených znaků (např. %, & či \), jejichž přítomnost v surové podobě by způsobila syntaktickou chybu a následné selhání kompilace.

Mikroslužba po přijetí požadavku provádí zpětné volání na endpointy backendu, aby získala potřebné zdrojové soubory. V případě požadavku na životopis vyučujícího nebo předmět se stahuje jeden konkrétní `.tex` soubor, zatímco u studijního programu mikroslužba postupně získává podklady (rovněž jako soubory s koncovkou `.tex`) pro všechny formuláře B-IIa (*Specializace*), B-III (*Předmět*) a C-I (*Vyučující*).

²Celý proces transformace je ve skutečnosti komplexnější a zahrnuje další podpůrné třídy. Pro zachování srozumitelnosti a plynulosti textu je však popsána pouze tato hlavní mechanika založená na mapování dat do šablon.

Získané soubory jsou uloženy do pracovního adresáře a následně je spuštěn proces kompilace. Výsledkem je buď samotný PDF dokument, nebo ZIP archiv obsahující kromě výsledného dokumentu i všechny zdrojové kódy, logy a podklady nutné pro lokální kompilaci. Shrnutím tedy je, že veškerá logika určující obsah a strukturu dat zůstává na backendu, zatímco mikroslužba plní pouze roli jejich kompletace a následné sazby.

Rozbor požadavků a návrh řešení

Tato kapitola blíže specifikuje požadavky práce a představuje teoretický návrh řešení. Popisuje nutné korekce stávajících výstupů, integraci chybějících akreditačních příloh a plán refaktorizace sázecí mikroslužby. Cílem je čtenáři přiblížit postup, který je následně aplikován při implementaci řešení.

5.1 Specifikace požadavků

Jak bylo rozebráno v kapitole 4, současný stav aplikace Akrmat vykazuje nedostatky, které brání jejímu efektivnímu využití pro akreditační účely. Cílem navržených úprav je vyřešit problémy spojené s chybějícími částmi akreditace i neúplným nebo nesprávným naplňováním šablon daty a uvést systém do stavu, kdy bude schopen generovat kompletní a formálně správné akreditační dokumenty.

Podkladem pro stanovení požadavků bylo porovnání aktuálních výstupů systému se vzorovými akreditačními materiály, které byly poskytnuty vedoucím práce ve formě finálního PDF dokumentu [25] a všech zdrojových souborů typu `.tex` použitých k jeho vytvoření. Analýza těchto rozdílů umožnila definovat konkrétní požadavky na rozšíření a úpravu systému, které jsou pro přehlednost rozděleny do následujících logických celků.

5.1.1 Korekce existujících výstupů

V rámci revize stávajících entit (*Vyučující*, *Předmět*, *Program*) byly stanoveny následující požadavky:

■ Vyučující:

- **Navigační prvky:** Doplnění hyperlinkových odkazů pro jména vyučujících napříč celým dokumentem. Toto rozšíření umožní navigaci mezi seznamy předmětů a konkrétními životopisy.
- **Název programu:** Doplnění názvu aktuálně akreditovaného studijního programu do výsledného formuláře C-I, který v současném řešení není uveden.
- **Výuka v jiných programech:** Zajištění kompletnosti tabulky dokumentující zapojení vyučujícího v rámci ostatních studijních programů (mimo aktuálně zpracováváný spis).

■ Předmět:

- **Aktualizace počtu parametrů šablony:** Přejít na novější verzi stylového souboru FITAK-25.sty vyžaduje přidání nového parametru TEACHING_METHODS (metody výuky), který nebyl v předchozích verzích nutný.
- **Dvojí reprezentace zapojení garanta:** Zavedení nového textového atributu pro slovní specifikaci formy zapojení garanta (např. „garant přednáší 100 % přednášek a zkouší“). Tento atribut doplňuje stávající číselnou hodnotu (procento zapojení), která je v systému využívána pro validaci celkového pokrytí výuky (suma zapojení všech vyučujících a garanta musí být 100).

■ Studijní program:

- **Deduplikace výstupů:** Zajištění integrity generovaného seznamu předmětů odstraněním případných duplicitních záznamů i v případě nekonzistentních dat v databázové vrstvě.
- **Revize formuláře B-IIa:** Úprava logiky generování seznamu vyučujících ke konkrétnímu předmětu v tomto formuláři tak, aby zobrazoval kompletní seznam vyučujících namísto pouze garanta předmětu. K těmto jménům dále přidat informaci o procentuálním podílu na výuce daného předmětu.

5.1.2 Rozšíření o nové formuláře

Hlavním požadavkem práce je doplnění podpory pro dosud neimplementované části akreditačního spisu. Konkrétně se jedná o administrativní přílohy A-I, B-I, personální a materiální přílohy C-II až C-V a textové přílohy D-I a E. Kromě

těchto standardizovaných formulářů je nutné systém rozšířit o generování titulní strany, úvodního textu a souhrnného seznamu garantů a vyučujících.

Pro tyto nové moduly (s výjimkou generovaného seznamu všech vyučujících, který bude vyžadovat úpravy pouze v aplikační a kompilační vrstvě) je nutné realizovat následující úpravy napříč všemi vrstvami aplikace:

- **Uživatelské rozhraní (Frontend):** Rozšíření klientské části o rozhraní, které správcům umožní vkládat a spravovat obsah nových příloh. U textových příloh D a E je nutné zachovat možnost plnohodnotného formátování textu (např. odrážky, tučné písmo). Zvolený způsob správy těchto formulářů proto musí tuto funkcionalitu uživateli poskytnout.
- **Aplikační logika (Backend):** Vytvoření nebo rozšíření příslušných servisních tříd a REST kontrolerů, které budou zajišťovat operace pro čtení, ukládání a úpravy formulářů. Tyto komponenty musí zároveň poskytovat data ve vhodné podobě pro následné zpracování sázecí službou.
- **Datová vrstva (Persistence):** Změna databázového schématu tak, aby umožnilo uložení dat těchto formulářů.
- **LaTeX mikroslužba:** Úprava logiky mikroslužby tak, aby nově definované formuláře a strukturní prvky (titulní strana, úvod, seznam vyučujících) byly zahrnuty do výsledného kompilačního procesu a zajistit jejich zobrazení ve finálním dokumentu PDF.

5.1.3 Další požadavky

Kromě předchozích požadavků na obsah akreditačního spisu byly na základě analýzy současného stavu aplikace a konzultací s vedoucím práce stanoveny další požadavky na kvalitu kódu a rozšíření systému:

- **RefaktORIZACE LaTeX mikroslužby:** Analýza stávajícího kódu mikroslužby pro generování výstupů odhalila značný technický dluh. Požadavkem je proto refaktORIZACE této služby s cílem zvýšit její modularitu a přehlednost. Součástí tohoto procesu je i zavedení automatizovaných testů.
- **Aktualizace stylového souboru:** Vzhledem k vývoji fakultních standardů je nutné provést migraci ze staršího stylového souboru `FITAK-24.sty` na aktuální verzi `FITAK-25.sty`. Tato změna vyžaduje drobné úpravy mapování dat do LaTeX maker tak, aby výsledné dokumenty splňovaly aktuální požadavky fakulty.
- **Rozšíření výstupních formátů:** V souladu s bodem 5 zadání, který se věnuje podpoře dalších datových formátů, byl po konzultaci zvolen formát Markdown. Tento formát bude implementován jako doplňkový výstup pro entitu *Předmět*. Markdown byl vybrán kvůli jeho čitelnosti a snadné

konverzi do dalších formátů, přičemž systém již v současnosti disponuje podporou pro JSON a HTML.

5.2 Návrh řešení

Na základě definovaných požadavků byl navržen postup pro jejich realizaci. Následující text popisuje plánované úpravy stávajících komponent, návrh generování nových dokumentů a nutnou refaktorizaci sázecí mikroslužby.

5.2.1 Úpravy stávajících výstupů

Úpravy stávajících formulářů budou spočívat primárně v revizi a rozšíření mapovací logiky v aplikační vrstvě. Většina změn se dotkne servisních tříd, které jsou zodpovědné za transformaci doménových objektů do formátu LaTeX (v architektuře aplikace jde typicky o komponenty s příponou `TexMapper`), případně statických `.tex` šablon, do kterých tyto mappery data doplňují. Tyto třídy budou v rámci implementace upraveny tak, aby výsledný zdrojový kód odpovídal nové verzi stylového souboru `FITAK-25.sty`, který v sázecí mikroslužbě nahradí původní `FITAK-24.sty`.

U předmětů a vyučujících budou doplněny chybějící parametry a bude implementována vnitřní navigace v dokumentu pomocí prvků `hyperlink`. Z hlediska datové perzistence dojde u existujících entit k jediné změně – přidání textového atributu u entity *Předmět* pro uložení slovní specifikace zapojení garanta. Ostatní logika, jako je například deduplikace seznamů předmětů, bude probíhat programově v rámci transformační logiky mapperů. Z hlediska celkové architektury se tedy tyto úpravy projeví téměř výhradně na backendové části aplikace. Uživatelské rozhraní nebude vyžadovat žádné změny a zásahy do databázového schématu či LaTeX mikroslužby budou minimální.

5.2.2 Návrh nových přehledů z existujících dat

Tato část návrhu se věnuje vytvoření nových výstupních dokumentů, které nevyžadují rozšíření datového modelu, neboť vycházejí z již existujících dat a datových vazeb.

Seznam vyučujících

Pro vytvoření personálního seznamu bude implementována agregační logika, která získá množinu vyučujících napříč všemi předměty studijního programu. Samotný výsledek ve formě tabulky bude realizován postupným mapováním, kde první úroveň zajistí formátování jednotlivých řádků s údaji o vyučujících, zatímco druhá úroveň sestaví řádky do finální tabulky. Tento proces bude v souladu s již existující logikou využívat statické šablony s placeholdery, které bude mapper dynamicky nahrazovat daty. Na úrovni aplikačního

rozhraní bude zároveň rozšířen stávající endpoint pro studijní programy, který nově kromě šablon předmětů, životopisů a specializací poskytne i šablonu pro seznam vyučujících. V rámci sázecí mikroslužby dojde k úpravě komunikačního mechanismu (*callbacku*), aby si služba o tento nový přehled automaticky žádala, přičemž upravena bude i hlavní sázecí šablona a další logika této služby tak, aby byl soubor se seznamem vyučujících zahrnut jak do kompilace výsledného PDF dokumentu, tak do balíčku zdrojových souborů pro účely ladění a případných manuálních úprav.

Markdown předmětu

Součástí návrhu je také export entity *Předmět* do formátu Markdown. Toto řešení bude využívat existující infrastrukturu pro nahrazování placeholderů v šablonách. Bude vytvořena samostatná mapovací komponenta a odpovídající statická šablona v syntaxi Markdown. Díky znovupoužití funkcionality pro zpracování šablon bude zajištěna transformace dat do nového formátu bez nutnosti vyvíjet další dodatečné mechanismy pro jejich manipulaci. Aby byl tento výstup dostupný uživateli, bude API endpoint pro entitu *Předmět* rozšířen o nový mediální typ `text/markdown` a v uživatelském rozhraní bude k předmětu přidáno tlačítko umožňující stažení dokumentu v tomto formátu. Navržený postup je aplikovatelný i na ostatní doménové entity systému. Případné budoucí rozšíření o další datové formáty tak bude vyžadovat pouze vytvoření specifické mapovací logiky a příslušných šablon bez nutnosti úprav již existujícího kódu aplikace, což činí toto řešení udržitelným a připraveným na další rozvoj.

5.2.3 Návrhy správy nových formulářů

Implementace nových akreditačních příloh a strukturních prvků rozšiřuje stávající systém. Všechny tyto entity jsou v rámci domény vázány na konkrétní studijní program. Návrh obsahuje dva uvážené koncepty technické realizace, které popisují následující podkapitoly spolu se zdůvodněním výběru.

5.2.3.1 Koncept fixního formulářového rozhraní

Tento koncept přistupuje k implementaci nových akreditačních příloh s využitím stávajících architektonických vzorů aplikace. Na úrovni datové vrstvy návrh počítá s vytvořením samostatných entit pro každý typ formuláře s pevnou vazbou na studijní program. Součástí řešení je implementace odpovídajících doménových tříd a servisní logiky na backendu, která zajišťuje validaci, ukládání a poskytování dat skrze aplikační rozhraní. V rámci uživatelského rozhraní tento přístup vyžaduje vývoj specifických editačních modulů a stránek přizpůsobených struktuře jednotlivých příloh. Tento vývoj je nutné realizovat pro každý nový typ formuláře, přičemž u textově orientovaných formulářů je nezbytné zajistit podporu formátování (např. odrážek či tučného písma) uvnitř

konkrétních polí. Pro proces generování dokumentace je nezbytné vytvořit nové mapovací komponenty typu **TexMapper** a statické šablony pro každý formulář zvlášť. Tyto komponenty transformují data z doménových entit do zdrojových souborů `.tex`, které je nutné zpřístupnit pomocí REST rozhraní. Na závěr proces vyžaduje úpravu LaTeX mikroslužby, aby tyto nové formuláře skrze endpointy backendu stahovala a zahrnovala je do výsledného kompilačního procesu celého akreditačního spisu.

5.2.3.2 Koncept přímé manipulace s obsahem

Druhý koncept přistupuje k řešení správy nových formulářů pomocí přímé manipulace s jejich zdrojovými kódy v systému LaTeX. Obsah těchto příloh je uložen jako textový řetězec přímo v rámci entity *Program*, což v databázové vrstvě vyžaduje rozšíření tabulky `programme` o příslušné sloupce typu `TEXT`. Aplikační logiku backendu je nutno rovněž upravit tak, aby umožňovala poskytování a aktualizaci těchto dat skrze rozhraní API. V uživatelském rozhraní je proces editace realizován formou stažení souboru typu `.tex`, jeho následnou lokální úpravou v libovolném textovém editoru dle výběru uživatele a opětovným nahráním upraveného dokumentu zpět do systému, kde je jeho obsah uložen do databáze. Jelikož data již v úložišti nabývají finální podoby zdrojového kódu LaTeX, není u tohoto řešení nutné vytvářet mapovací třídy typu **TexMapper** ani definovat statické šablony. Pro zajištění kompletnosti akreditačního spisu je i u tohoto konceptu nezbytné implementovat rozhraní, které tyto uložené textové bloky poskytne LaTeX mikroslužbě. V neposlední řadě je nutné upravit logiku samotné mikroslužby tak, aby textové bloky skrze nové endpointy backendu stahovala a zahrnula je do výsledného kompilačního procesu celého akreditačního spisu.

5.2.3.3 Diskuse a zdůvodnění výběru

Při srovnání obou konceptů vykazuje první varianta násobně vyšší implementační náročnost a v budoucnu by byla její údržba komplikovanější. Jakákoliv změna v požadavcích na strukturu formulářů by v tomto případě vyžadovala úpravy napříč všemi vrstvami aplikace, od databázového schématu přes backendovou logiku až po komponenty uživatelského rozhraní. Flexibilní řešení naproti tomu umožňuje provádět změny pouze úpravou textových dat, případně nahráním nové verze stylového souboru s aktualizovaným makrem, což proces údržby zjednodušuje.

Nevýhodou flexibilního přístupu je absence validace dat a mechanismů pro jejich synchronizaci. Pokud uživatel v textovém obsahu nedodrží syntaktická pravidla systému LaTeX, například neuzavřením závorky argumentu, dojde k selhání kompilace. Tento problém je částečně zmírněn poskytováním ZIP archivu pro účely ladění, přesto však koncept vyžaduje od uživatele znalost systému LaTeX a struktury používaných maker. S přímým vstupem uživatele

ve formě zdrojového kódu souvisí také teoretické riziko útoku typu *LaTeX injection* například v podobě příkazu `\input{/etc/passwd}` [26]. V kontextu systému Akmat to však nepředstavuje bezpečnostní hrozbu, protože přístup k editačním funkcím je omezen výhradně na prověřené pracovníky fakulty a také protože je mikroslužba spuštěna v izolovaném prostředí Docker kontejneru. Absence automatického doplňování údajů z jiných částí systému rovněž nepředstavuje u těchto formulářů překážku, neboť přílohy obsahují data specifická pro daný program, která se v rámci akreditačního spisu dále neopakují. Počáteční manuální vyplnění je tedy nezbytné u obou zvažovaných konceptů.

Fixní rozhraní se dále jeví jako nevhodné zejména pro přílohu typu E, která nemá vždy stejnou strukturu. Pokus o definici pevného rozhraní pro takový dokument by mohl vyústit v existenci velkého množství nevyužitých databázových polí nebo v absenci atributů, jejichž potřeba by vyvstala až v pozdějších fázích provozu. Flexibilní řešení těmito omezeními netrpí a umožňuje dynamicky přizpůsobovat obsah dokumentu aktuálním potřebám bez nutnosti zásahu do systému.

Výhody a nevýhody obou přístupů byly pečlivě zváženy a po konzultaci s vedoucím práce bylo rozhodnuto pro realizaci konceptu přímé manipulace s obsahem. Vzhledem k tomu, že správce aplikace disponuje potřebnými znalostmi systému LaTeX i struktury definovaných maker, jsou celkové dopady a přínosy tohoto řešení, zejména s ohledem na dlouhodobou flexibilitu správy dokumentace, vhodnější než v případě fixního rozhraní.

5.2.4 Návrh refaktORIZACE mikroslužby

Návrh refaktORIZACE mikroslužby řeší identifikovaný technický dluh. Hlavním bodem je dekompozice třídy `ApiController` do samostatných servisních komponent, což odstraní problém s duplicitou kódu a zlepší jeho udržitelnost. Riziko *race condition* při souběžných požadavcích bude řešeno zavedením mechanismu izolace pracovních adresářů.

Proces kompilace bude rozšířen o podporu vícenásobných průchodů, čímž bude zajištěna správná sazba křížových odkazů, která v současném řešení nefunguje správně. Součástí úprav je také aktualizace stylového souboru na aktuální variantu `FITAK-25.sty` a doplnění sady automatizovaných testů pro ověření stability systému při zavádění změn.

Implementace

Tato kapitola se věnuje realizaci navrženého řešení. Jejím účelem je seznámit čtenáře s konkrétními technickými úpravami napříč celou architekturou aplikace a způsobem, jakým byly definované požadavky prakticky implementovány.

6.1 Rozšíření datového úložiště

Na základě požadavků bylo nutné rozšířit stávající databázové schéma. Veškeré změny v databázové vrstvě jsou v systému Akrmat verzovány pomocí nástroje Flyway, který zajišťuje automatickou a deterministickou migraci struktury databáze napříč všemi prostředími.

Pro splnění požadavku na implementaci konceptu přímé manipulace s obsahem u nových akreditačních formulářů byla rozšířena tabulka **programme**. K této entitě byly přidány nové sloupce datového typu **TEXT**, které slouží pro uložení zdrojových kódů systému LaTeX pro příslušné přílohy. V případě entity *Předmět* došlo k přidání sloupce pro uložení textové specifikace zapojení garanta do výuky, čímž byl splněn požadavek na textovou reprezentaci tohoto údaje.

Všechny nově přidávané sloupce byly v migračním skriptu definovány bez omezení NOT NULL (jsou tedy tzv. *nullable*). To umožňuje zpětnou kompatibilitu se stávajícími daty v systému, a není tedy nutné vytvářet dodatečné migrační skripty pro vyplnění výchozích hodnot u již existujících záznamů studijních programů a předmětů, které tyto nové údaje pochopitelně zatím neobsahují.

V souladu s principy nástroje Flyway nelze modifikovat již aplikované historické skripty. Pro zachování konzistence databáze je tak nutné veškeré změny realizovat formou nových, inkrementálních skriptů. Následující ukázka kódu ilustruje podobu Flyway migračních skriptů (napsaných v jazyce SQL), které tyto změny v databázovém schématu realizují:

```
1  -- Skript V5__add_tex_form_columns_to_study_program.sql
2  ALTER TABLE akrmat.programme
3  ADD COLUMN title_tex TEXT,
4  ADD COLUMN introduction_tex TEXT,
5  ADD COLUMN form_ai_tex TEXT,
6  ADD COLUMN form_bi_tex TEXT,
7  ADD COLUMN form_cii_tex TEXT,
8  ADD COLUMN form_ciii_tex TEXT,
9  ADD COLUMN form_civ_tex TEXT,
10 ADD COLUMN form_cv_tex TEXT,
11 ADD COLUMN form_di_tex TEXT,
12 ADD COLUMN attachment_ei_tex TEXT;
13
14 -- Skript V6__add_guarantor_involvement_str.sql
15 ALTER TABLE akrmat.course
16 ADD COLUMN guarantor_involvement_string VARCHAR(100);
```

■ **Výpis kódu 3** Migrační skripty pro rozšíření tabulek programme a course

6.2 Rozvoj aplikační logiky

Tato sekce se věnuje úpravám serverové části aplikace. Následující podkapitoly postupně představují rozšíření doménového modelu, revizi existující aplikační logiky pro generování dokumentů, zavedení exportu do formátu Markdown a s tím související aktualizaci aplikačního rozhraní.

6.2.1 Rozšíření doménového modelu

Pro implementaci nových akreditačních formulářů a strukturních prvků do aplikační vrstvy bylo nutné nejprve rozšířit samotné doménové modely a ORM entity o příslušné textové atributy. Vytvoření samostatných servisních metod a API endpointů pro každý z deseti nových formulářů by nevyhnutelně vedlo k duplicitě kódu a špatné udržitelnosti. Místo toho byl využit polymorfismus k vytvoření univerzálního řešení pomocí výčtového typu (`enum`) `TexSection`.

Ten mapuje textové identifikátory (využívané v URL adresách na straně API) přímo na konkrétní akreditační přílohy. Každá konstanta uvnitř enumerace implementuje abstraktní metody `get` a `set`, které definují přístup k příslušnému atributu entity *Program*. Tímto přístupem je veškerá logika pro směrování dat zapouzdřena přímo v doménovém modelu, díky čemuž není nutné využívat rozsáhlé rozhodovací bloky.

Přínosem tohoto řešení je, že kód je snadno rozšiřitelný a současně se budoucí změny projeví pouze v omezeném rozsahu. Jak ukazuje následující výpis kódu, servisní třída využívá v tomto řešení jednu univerzální metodu pro čtení

a jednu pro zápis. Pokud by v budoucnu vyvstala potřeba přidat do systému další typ formuláře využívajícího tento koncept přímé manipulace s obsahem, stačí v rámci aplikační logiky pouze přidat novou konstantu do výčtového typu a upravit odpovídající doménové entity. Servisní vrstva i REST kontrolery zůstanou zcela beze změny, jelikož systém novou sekci rozpozná a obsluží.

```

1  enum class TexSection(val urlKey: String) {
2      TITLE("title") {
3          override fun get(p: Programme) = p.titleTex
4          override fun set(p: Programme, v: String?)
5              { p.titleTex = v }
6      },
7
8      // ... (zkráceno pro přehlednost) ...
9
10     ATTACHMENTEI("attachmentEI") {
11         override fun get(p: Programme) = p.attachmentEITex
12         override fun set(p: Programme, v: String?)
13             { p.attachmentEITex = v }
14     };
15
16     abstract fun get(p: Programme): String?
17     abstract fun set(p: Programme, v: String?)
18
19     companion object {
20         // Transform a string key to enum or throws error
21         fun fromUrlKey(key: String): TexSection =
22             values().find { it.urlKey.equals(key, ignoreCase = true) }
23             ?: throw IllegalArgumentException(
24                 ".tex section '$key' is not supported/existing."
25             )
26     }
27 }
28
29 // ukázka využití v servisní třídě
30
31 @Transactional
32 override fun updateTexContent(
33     programmeCode: String, section: TexSection, content: String
34 ) {
35     val programme = findByCode(programmeCode)
36     ?: throw NotFound("Programme $programmeCode not found")
37
38     // utilize the section polymorphism
39     section.set(programme, content)
40
41     save(programme)
42 }
43

```

■ Výpis kódu 4 Definice výčtového typu a ukázka využití

6.2.2 Revize mapovacích komponent

Část úprav aplikační logiky spočívala v revizi a opravě stávajících komponent typu `TexMapper`. Tyto třídy jsou zodpovědné za transformaci doménových objektů do podoby zdrojového kódu LaTeX, respektive za nahrazování zástupných textů (*placeholders*) ve statických šablonách reálnými daty. Cílem revize bylo odstranit chyby (např. chybějící parametry či nesprávné mapování atributů) původního řešení a zajistit shodu generovaných výstupů s aktuálními požadavky fakulty. Mezi nejvýznamnější realizované úpravy patří:

- **Aktualizace pro nový stylový soubor:** Přizpůsobení mapovací logiky syntaxi a makrům definovaným ve verzi `FITAK-25.sty`. To zahrnovalo například zavedení nového parametru pro metody výuky nebo úpravu struktury předávaných dat u dalších formulářů.
- **Korekce dat:** Implementace deduplikační logiky při generování formulářů předmětů a vyučujících, která zajišťuje korektní výstup i v případě duplicitních záznamů v databázi. Dále byla rozšířena logika pro sběr dat o vyučujících tak, aby zahrnovala informace o jejich působení v jiných studijních programech.
- **Doplnění chybějících prvků:** Do příslušných částí šablon byla přidána podpora pro křížové odkazy a hyperlinky. Zavedena byla také duální reprezentace zapojení garanta předmětu a propisování názvu aktuálního programu do personálních příloh vyučujících.

6.2.3 Generování souhrnného seznamu vyučujících

Specifickým požadavkem byla implementace logiky pro agregaci personálního zabezpečení ve formě samostatného seznamu napříč celým studijním programem. Tato funkčnost byla realizována v rámci servisní vrstvy, kde jsou rekurzivně sbírána data o vyučujících z hlavního studijního plánu i ze všech přidružených specializací.

V rámci servisní třídy byla vytvořena metoda `createTeacherListTex`, která zajišťuje očištění dat a jejich přípravu pro sázecí mikroslužbu. Pro generování obsahu tabulky byla rozšířena komponenta `TeacherTexMapping` o metodu `getTeacherTableRowPlaceholders`, která využívá nově vytvořenou statickou šablonu pro definici vizuální podoby jednoho řádku.

Pro mapování řádků byla implementována logika, která pro každého z vyučujících filtruje seznam vyučovaných předmětů. Výsledný dokument tedy u vyučujících obsahuje pouze ty předměty, které jsou relevantní pro aktuálně generovaný akreditační spis. Tato data jsou následně poskytována sázecí mikroslužbě skrze parametrizovaný endpoint aplikačního rozhraní, stejným způsobem jako u *Předmětů*, *Vyučujících* a *Specializací*.


```
1 \tbunka{\hyperlink{USERNAME}-{SURNAME}} & % prijmeni s kotvou
2 \tbunka{NAME} & % jmeno
3 \tbunka{TITLES} & % tituly
4 \tbunka{VS_EXTENT} & % na VS
5 \tbunka{FACULTY_EXTENT} & % na soucasti
6 \tbunka{COURSES} & % predmety
7 \tbunka{PRACTICE_EXPERT} & % odbornik z praxe
8 \tbunka{UNIVERSITY} \\ % VS
9 \hline
```

■ **Výpis kódu 5** Šablona pro řádek seznamu vyučujících

6.2.4 Export do formátu Markdown

Systém byl rozšířen o export dat entity *Předmět* do formátu Markdown, k čemuž byl v REST API zaveden mediální typ `text/markdown`. Hlavním bodem implementace je nová mapovací třída `CourseMarkdownMapping`, která zajišťuje přípravu dat, ošetření speciálních znaků syntaxe Markdown a vytvoření mapování (dvojic) mezi zástupnými texty (*placeholders*) a reálnými hodnotami.

```
1 // Zjednodušená ukázka mapovací třídy
2 object CourseMarkdownMapping {
3     fun getPlaceholders(course: Course) : Map<String, String>
4     {
5         val placeholders = mutableMapOf<String, String>()
6
7         placeholders["COURSE_NAME"] = course.name
8         placeholders["COURSE_ABBREVIATION"] = course.code
9         placeholders["CREDIT_COUNT"] = course.creditAmount?.toString()
10        // ... (další atributy vynechány pro přehlednost)
11
12        return placeholders
13    }
14 }
15
16 // Servisní metoda orchestrující spojení dat a šablony
17 @PreAuthorize("hasAnyRole('$READER', '$EDITOR', '$ADMIN')")
18 fun generateMarkdown(courseCode: String, formName: String) : String
19 {
20     val course = findByCode(courseCode)
21     val placeholders = CourseMarkdownMapping.getPlaceholders(course)
22
23     return formGeneratorService
24         .generate(FULL_COURSE_MD_PATH, placeholders)
25 }
```

■ **Výpis kódu 6** Ukázka CourseMarkdownMapping objektu a jeho použití

Dále byla vytvořena nová statická šablona `course.md`. Samotné generování obsahu probíhá pomocí již existující služby `FormGeneratorService`, která připravené dvojice převeze a provede samotné nahrazení zástupných textů v šabloně. Toto řešení je tak v souladu se stávající architekturou aplikace a znovupoužívá již přítomné mechanismy. Díky oddělení přípravy dat od jejich vizuální reprezentace stačí pro budoucí úpravy formátování modifikovat pouze statickou šablonu, zatímco začlenění nových údajů do finálního výsledku vyžaduje pouze úpravu mapperu a šablony. Výše popsáný postup je navíc univerzální – případné rozšíření systému o další výstupní formáty (např. YAML) lze v budoucnu realizovat obdobným postupem.

```

1 # COURSE_NAME (COURSE_ABBREVIATION)
2
3 ## Základní informace
4 | Vlastnost | Hodnota |
5 | :--- | :--- |
6 | **Kredity** | CREDIT_COUNT |
7 | **Zakončení** | COMPLETION |
8
9 ## Personální zabezpečení
10 | Role | Jméno | Poznámka / Podíl |
11 | :--- | :--- | :--- |
12 | **Garant** | GUARANTORS | GUARANTOR_INVOLVEMENT |
13 ...

```

■ **Výpis kódu 7** Ukázka začátku Markdown šablony

6.2.5 Aktualizace aplikačního rozhraní

Aby bylo možné nově implementovanou aplikační logiku využívat ze strany klientské aplikace i sázecí mikroslužby, bylo nezbytné stávající aplikační rozhraní doplnit o nové endpointy. Pro manipulaci s novými akreditačními přílohami byl na úrovni kontrolerů využit dříve představený polymorfní výčtový typ `TexSection`. To umožnilo vytvořit pouze jeden parametrizovaný endpoint pro čtení a jeden pro zápis, které obsluhují všechny nově přidané sekce programu.

Dále byl implementován specifický endpoint poskytující všechny nově přidané akreditační přílohy hromadně ve formátu ZIP. Tento koncový bod je určen výhradně pro sázecí mikroslužbu a umožňuje stáhnout potřebné textové podklady v rámci jednoho síťového požadavku.

■ **Tabulka 6.1** Přehled přidáných endpointů

Metoda	URL	Akce
GET	/programmes/{programmeId}/tex/{section}	Stahování .tex formuláře
POST	/programmes/{programmeId}/tex/{section}	Nahrání .tex formuláře
GET	/programmes/{programmeId}/tex-bundle	Stahování ZIP archivu formulářů

6.3 Refaktorizace a rozšíření LaTeX mikroslužby

Úpravy sázecí mikroslužby zahrnovaly jak rozšíření její funkcionality o zpracování nových formulářů, tak nezbytnou refaktorizaci stávajícího zdrojového kódu. Následující text popisuje provedené strukturální změny, řešení problému se sdíleným pracovním adresářem a úpravy samotného kompilačního procesu.

6.3.1 Strukturální změny aplikace

V původní verzi mikroslužby byla téměř veškerá logika soustředěna přímo v kontroleru, což vedlo k duplicitě kódu a špatné udržitelnosti. V rámci refaktorizace byla vytvořena nová servisní vrstva realizovaná třídou `LatexService`. Do této třídy byla extrahována opakující se doménová logika, jako je stahování zdrojových souborů z backendu, manipulace se ZIP archivy a samotné spouštění systémových příkazů pro kompilaci. Kontroler (`ApiController`) si tak nově ponechal pouze roli orchestrátoru.

Dalším krokem bylo odstranění pevně zakódovaného (tzv. hardcoded) testovacího přístupového tokenu přímo ze zdrojového kódu. Ačkoliv mikroslužba prozatím stále využívá testovací token pro základní ověřování, byl přesunut do konfiguračního souboru `application.properties`. Při budoucím nasazení tak lze tento token snadno nahradit plně funkčním tokenem bez nutnosti zásahu do kódu služby.

6.3.2 Izolace pracovních adresářů

Jedním z rizik dřívější implementace byla možnost vzniku chyb typu *race condition* při souběžném zpracování požadavků od více uživatelů, jelikož služba využívala jeden sdílený pracovní adresář. Tento nedostatek byl vyřešen zavedením mechanismu dynamické tvorby dočasných adresářů.

Služba `LatexService` vytvoří při každém novém požadavku na generování dokumentu unikátní pracovní adresář identifikovaný pomocí náhodně vygenerovaného řetězce UUID. Do tohoto izolovaného prostředí jsou následně zkopírovány všechny potřebné statické soubory (jako je například stylový soubor `FITAK-25.sty`) a následně i stažené dynamické podklady od backendu. Po dokončení kompilace a odeslání výsledku uživateli je tento dočasný adresář smazán metodou `cleanup`. Ačkoliv kopírování statických souborů nepředstavuje z hlediska výkonu nejefektivnější možné řešení, v kontextu aplikace Akrmat je tento přístup plně dostačující. Generování akreditačních materiálů nepředstavuje časově kritickou operaci a z hlediska zátěže systému probíhá spíše nárazově, proto byla izolace procesů upřednostněna před optimalizací výkonu.

6.3.3 Kompilační proces

Pro správné vytvoření křížových odkazů a hyperlinků byl kompilační proces upraven tak, že mikroslužba spouští příkaz `pdflatex` nad zdrojovými soubory dvakrát po sobě, čímž je zajištěna funkční navigace ve výsledném dokumentu. Zároveň byl aktualizován obraz Docker kontejneru o dodatečné jazykové balíčky, což zaručuje mimo jiné například korektní sazbu kalendářních dat.

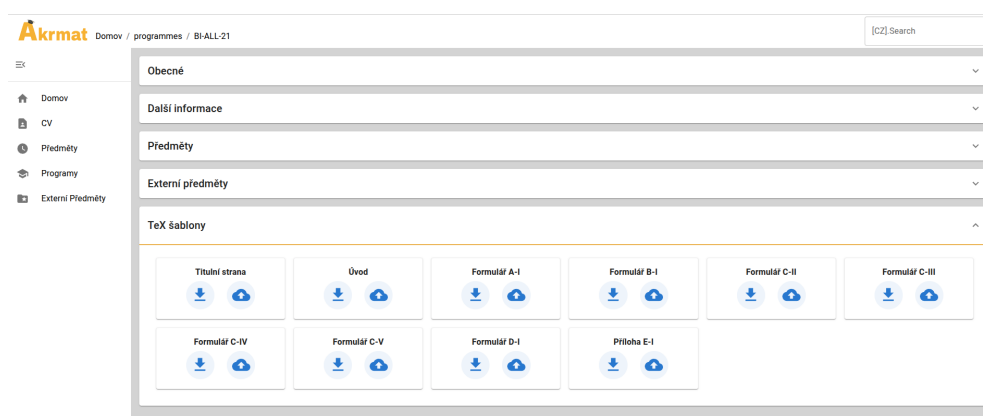
Nutným rozšířením z hlediska obsahu je implementace logiky pro začlenění všech nově vytvořených akreditačních příloh do výsledného akreditačního spisu. Mikroslužba si před samotnou kompilací vyžádá z backendu ZIP archiv

obsahující všechny tyto nové formuláře ve formě souborů s příponou `.tex`. Tento balíček následně rozbalí do izolovaného pracovního adresáře a získané soubory zahrne jak do hlavní kompilace výsledného PDF, tak do exportu archivu určeného pro případné manuální úpravy.

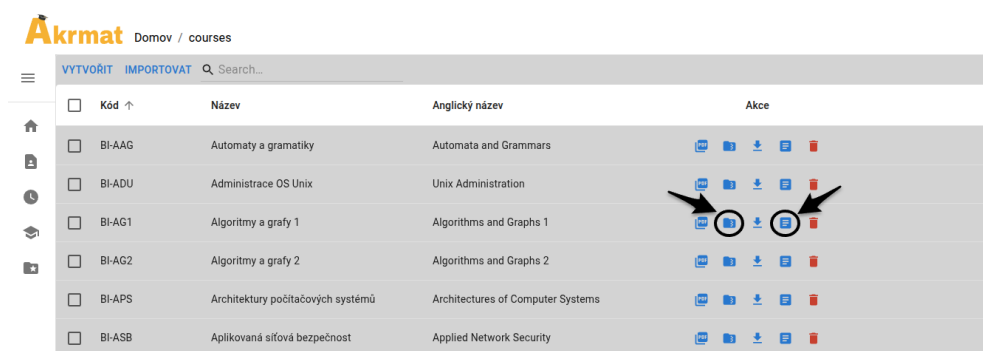
6.4 Změny v uživatelském rozhraní

Aby bylo možné využívat nově implementované funkcionality backendu a sázecí mikroslužby, bylo nezbytné odpovídajícím způsobem rozšířit i klientskou část aplikace. Úpravy uživatelského rozhraní spočívaly především v tvorbě a umístění nových ovládacích prvků do stávajících pohledů a jejich napojení na aplikační rozhraní backendu. Konkrétně se jedná o následující změny:

- **Správa akreditačních příloh:** V detailu studijního programu byly přidány ovládací prvky pro práci s novými formuláři, které uživateli umožňují stáhnout nebo nahrát konkrétní část ve formátu `.tex` (viz Obrázek 6.1).
- **Alternativní export předmětu:** U studijního předmětu bylo k existujícím možnostem exportu přidáno nové tlačítko, které vyvolá vygenerování a následné stažení sylabu předmětu ve formátu Markdown. (viz Obrázek 6.2).
- **Export ZIP archivu:** K entitám: *Vyučující*, *Předmět* a *Studijní program* byly doplněny ovládací prvky umožňující stažení kompletního ZIP archivu s podklady pro kompilaci. Ačkoliv byla tato funkcionality na straně backendu již podporována, v uživatelském rozhraní dosud chyběla možnost jejího využití (viz Obrázek 6.2).



■ **Obrázek 6.1** Ukázka ovládacích prvků nových formulářů



Obrázek 6.2 Ukázka ovládacích prvků pro nové exporty

6.5 Testování a dokumentace

Součástí implementace bylo také otestování nově přidáných funkcí a vytvoření příslušné technické dokumentace. Nový zdrojový kód byl opatřen komentáři, pro aplikační rozhraní byla využita automaticky generovaná dokumentace a správnost implementované logiky byla ověřena sadou automatizovaných testů.

6.5.1 Dokumentace

Během vývoje byl dodržován princip samodokumentujícího kódu, který usnadňuje čitelnost díky vhodně zvoleným názvům tříd, metod a proměnných. Složitější části nově vytvořeného či refaktorovaného kódu byly zdokumentovány přímo ve zdrojových souborech pomocí komentářů. Pro backendovou část napsanou v jazyce Kotlin byl využit standard KDoc, zatímco v refaktorované mikroslužbě byl použit standard Javadoc. Tato interní dokumentace se zaměřuje především na logiku mapování dat pro LaTeX a Markdown šablony, zpracování jednotlivých sekcí akreditačních materiálů a integrační vrstvy mezi doménovým modelem a API. V rámci mikroslužby byl rovněž aktualizován soubor README, který popisuje její současné závislosti a aplikační rozhraní.

Pro dokumentaci aplikačního rozhraní backendu byla využita knihovna SpringDoc, která dynamicky generuje specifikaci podle standardu OpenAPI 3. Díky tomu je technická dokumentace všech endpointů vždy aktuální a dostupná přes interaktivní rozhraní *Swagger UI*. Toto rozhraní lze zároveň použít jako nástroj pro manuální testování vyvinutých endpointů během vývoje [27].

6.5.2 Testování

Ověření správné funkčnosti řešení bylo zajištěno pomocí automatizovaných jednotkových testů. Tyto testy byly přidány jak pro nové části aplikační logiky na straně backendu, tak pro sázecí mikroslužbu. Jelikož ta v původní verzi zcela

postrádala jakékoliv testování, byly vytvořeny nové testovací scénáře tak, aby pokryly refaktorované třídy a extrahovanou servisní logiku.

Veškeré tyto testy jsou integrovány do fakultního systému GitLab, kde se automaticky spouštějí v rámci CI/CD pipeline při každé změně zdrojového kódu. Je nutné podotknout, že v rámci této práce byly vytvořeny pouze jednotkové testy. Tvorba end-to-end testů nebyla předmětem tohoto řešení, neboť se této problematice pro celý systém Akrmat věnuje souběžně vznikající bakalářská práce Matúše Walla.

```
1  @Test
2  fun `should correctly set and get introduction tex using enum`() {
3      val programme = TestEntities.programmeDomain()
4      val section = TexSection.INTRO
5      val testContent = "\\section{Úvod}"
6
7      section.set(programme, testContent)
8      val result = section.get(programme)
9
10     assertEquals(testContent, programme.introductionTex)
11     assertEquals(testContent, result)
12 }
```

■ **Výpis kódu 8** Ukázka testu výčtového typu



Kapitola 7

Závěr

Hlavním cílem této práce bylo rozšířit novou generaci informačního systému Akrmat o podporu dosud neimplementovaných akreditačních formulářů a upravit předchozí logiku aplikace tak, aby byl systém schopen samostatně generovat kompletní a formálně správné akreditační spisy. Výsledkem této práce je funkční řešení, které tento cíl naplňuje.

V rámci práce byla provedena analýza současného stavu aplikace, na jejímž základě byl navržen a následně implementován vhodný způsob správy nových dokumentů spolu s nezbytnou revizí a rozšířením stávající aplikační logiky. Úpravy se dotkly všech vrstev systému – od rozšíření databázového schématu, přes implementaci nové aplikační logiky na backendu a přidání ovládacích prvků do uživatelského rozhraní, až po úpravu kompilačního procesu. Během analýzy a vývoje navíc vyvstala potřeba řešit existující technický dluh, což vedlo k dodatečnému, avšak nezbytnému refaktoringu LaTeX mikroslužby. K existujícím exportům byla také přidána podpora formátu Markdown, čímž systém nabízí další způsob, jak data entity *Předmět* reprezentovat.

Díky provedeným úpravám je systém Akrmat po stránce generování akreditačního spisu připraven nahradit svou původní verzi. Mechanismus generování materiálů nyní produkuje výstupy odpovídající aktuálním požadavkům fakulty. Aby však bylo možné aplikaci naplno využívat pro podávání akreditací, je nezbytným budoucím krokem naplnění databáze reálnými a konzistentními daty. Datová sada poskytnutá a využívaná v průběhu vývoje nebyla úplná, avšak jakmile správce systému chybějící informace do databáze doplní, bude systém generovat hotové dokumenty.

Během vývoje byly rozpoznány i další možnosti rozvoje aplikace, které však již přesahují rámec zadání této práce. Vzhledem ke zvolenému konceptu přímé manipulace se zdrojovými kódy by v budoucnu bylo přínosné implementovat syntaktickou validaci uživatelského vstupu přímo do uživatelského rozhraní. Systém by mohl automaticky kontrolovat správnost syntaxe LaTeX kódu, například validovat počet parametrů volaných maker či integritu složených závorek, a doplnit tyto kontroly o pokročilejší ochranu proti útokům

typu *LaTeX injection*. Dalším možným vylepšením usnadňujícím dlouhodobou údržbu by mohlo být vytvoření rozhraní pro přímé nahrávání a aktualizaci stylového souboru (např. budoucí verze **FITAK-26.sty**) z webové aplikace.

Všechny dílčí body zadání byly úspěšně splněny. Výsledkem je funkční a zdokumentované řešení, které správci aplikace umožňuje generovat kompletní akreditační materiály bez nutnosti dřívějších dodatečných korekcí. Realizované změny tak usnadňují a zpřehledňují celý proces přípravy akreditačních podkladů na fakultě.

A decorative horizontal bar consisting of approximately 20 small, light-blue squares arranged in a single row.

Zdrojové kódy

Zdrojové kódy ke všem částem práce jsou dostupné na fakultním vývojovém prostředí *GitLab* na adrese <https://gitlab.fit.cvut.cz/>. Veškeré úpravy a nově implementované funkce v rámci této práce jsou v příslušných repozitářích dostupné v samostatné vývojové větvi s názvem `jelinkry-thesis`

- **Backend** – zdrojové kódy backendové části aplikace jsou dostupné na adrese <https://gitlab.fit.cvut.cz/akrmat/akrmat-v4-backend>.
- **Frontend** – zdrojové kódy frontendové části aplikace jsou dostupné na adrese <https://gitlab.fit.cvut.cz/akrmat/akrmat-frontend>.
- **LaTeX mikroslužba** – zdrojové kódy k sázecí mikroslužbě jsou dostupné na adrese <https://gitlab.fit.cvut.cz/akrmat/akrmat-latex>.

Bibliografie

1. ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE. *Akreditační řád Českého vysokého učení technického v Praze* [Vnitřní předpis, online]. Praha, 2021 [cit. 2026-03-14]. Dostupné z: <https://www.cvut.cz/vnit-rni-predpisy>.
2. NÁRODNÍ AKREDITAČNÍ ÚŘAD PRO VYSOKÉ ŠKOLSTVÍ. *Doporučené postupy pro přípravu studijních programů* [Metodický materiál, online]. 2024. [cit. 2026-03-14]. Dostupné z: <https://nauvs.cz/index.php/cs/metodiky/79-doporucene-postupy-pro-pripravu-studijnic-h-programu>.
3. NÁRODNÍ AKREDITAČNÍ ÚŘAD PRO VYSOKÉ ŠKOLSTVÍ. *Bližší informace a pokyny k údajům požadovaným v přílohách A-D žádosti o akreditaci studijního programu* [Metodický materiál, online]. 2024. [cit. 2026-03-14]. Dostupné z: <https://www.nauvs.cz/index.php/cs/metodiky/71-metodicke-materialy-pro-pripravu-a-hodnoceni-zadosti-o-akreditaci-studijniho-programu>.
4. NÁRODNÍ AKREDITAČNÍ ÚŘAD PRO VYSOKÉ ŠKOLSTVÍ. *Doporučená osnova sebehodnotící zprávy pro akreditaci studijních programů (podklad pro přílohu E)* [Metodický materiál, online]. 2022. [cit. 2026-03-14]. Dostupné z: <https://www.nauvs.cz/index.php/cs/metodiky/71-metodicke-materialy-pro-pripravu-a-hodnoceni-zadosti-o-akreditaci-studijniho-programu>.
5. MINISTERSTVO VNITRA ČR. *e-Sbírka: Elektronická Sbírka zákonů a mezinárodních smluv* [online]. 2026. [cit. 2026-04-14]. Dostupné z: <https://e-sbirka.gov.cz/>.
6. META PLATFORMS, INC. *React* [online]. 2026. [cit. 2026-03-10]. Dostupné z: <https://react.dev/>.
7. MICROSOFT. *TypeScript Documentation* [online]. Microsoft, 2026 [cit. 2026-03-12]. Dostupné z: <https://www.typescriptlang.org/docs/>.

8. AKHIN, Marat; BELYAEV, Mikhail et al. *Kotlin Language Specification: Kotlin/Core* [online]. JetBrains / JetBrains Research, 2020. [cit. 2026-03-12]. Dostupné z: <https://kotlinlang.org/spec/introduction.html>.
9. GEEKSFORGEES. *Introduction to Spring Framework* [online]. 2026. [cit. 2026-03-12]. Dostupné z: <https://www.geeksforgeeks.org/introduction-to-spring-framework/>.
10. MICROSOFT. *What is Java Spring Boot?* [online]. 2026. [cit. 2026-03-12]. Dostupné z: <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-java-spring-boot>.
11. GEEKSFORGEES. *Spring Cloud – OpenFeign with Example Project* [online]. 2025. [cit. 2026-03-12]. Dostupné z: <https://www.geeksforgeeks.org/java/spring-cloud-openfeign-with-example-project/>.
12. BAUER, Christian; KING, Gavin. *Hibernate in Action*. Greenwich, CT: Manning Publications Co., 2005. ISBN 1932394-15-X.
13. OBREGON, Alexander. *Spring Boot and JPA Integration Explained* [online]. Medium, 2024 [cit. 2026-03-12]. Dostupné z: <https://medium.com/@AlexanderObregon/spring-boot-and-jpa-integration-explained-dac36400889b>.
14. GUPTA, Lokesh. *What is REST?* [online]. 2025. [cit. 2026-03-13]. Dostupné z: <https://restfulapi.net/>.
15. MONGODB INC. *What Is an Object-Oriented Database?* [online]. 2026. [cit. 2026-03-13]. Dostupné z: <https://www.mongodb.com/resources/basics/databases/what-is-an-object-oriented-database>.
16. SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. *Database System Concepts*. 7th. McGraw-Hill Education, 2020. ISBN 978-0-07-802215-9.
17. BAELDUNG. *Database Migrations with Flyway* [online]. 2026. [cit. 2026-03-13]. Dostupné z: <https://www.baeldung.com/database-migrations-with-flyway>.
18. URBAN, Michael. *An Introduction to L^AT_EX* [online]. 2019. [cit. 2026-03-14]. Technical Report. TRW Software Productivity Project. Dostupné z: <https://www.tug.org/texniques/tn09/intro.pdf>.
19. DOCKER INC. *Orientation and setup* [online]. 2020. [cit. 2026-03-14]. Dostupné z: <https://docker-docs.uclv.cu/get-started/>.
20. SOFTWARE MIND. *What is a Single Page Application (SPA)?* [online]. 2024. [cit. 2026-03-15]. Dostupné z: <https://softwaremind.com/blog/what-is-a-single-page-application-spa/>.
21. MUI. *Material UI: Getting started* [online]. MUI, 2026. [cit. 2026-03-15]. Dostupné z: <https://mui.com/material-ui/getting-started/>.

22. ŠTEFÁNIK, Jan. *Aplikace Akrmát – verze 4.0 – backend* [online]. Praha, 2023 [cit. 2026-03-10]. Dostupné z: <http://hdl.handle.net/10467/108785>. Diplomová práce. České vysoké učení technické v Praze, Fakulta informačních technologií.
23. KRAJÍČEK, Zdeněk. *Optimalizace datového úložiště a backend webové aplikace Akrmát* [online]. Praha, 2026 [cit. 2026-03-10]. Dostupné z: <https://hdl.handle.net/10467/179376>. Bakalářská práce. České vysoké učení technické v Praze, Fakulta informačních technologií.
24. NAUMOV, Alexey. *Separation of Concerns in Software Design* [online]. 2020. [cit. 2026-03-17]. Dostupné z: <https://nalexn.github.io/separation-of-concerns/>.
25. ČVUT V PRAZE, FAKULTA INFORMAČNÍCH TECHNOLOGIÍ. *Žádost o prodloužení platnosti akreditace části magisterského programu Informatika jako Aplikovaná informatika a jeho rozšíření o jednu specializaci*. Praha, 2025. Akreditační spis. České vysoké učení technické v Praze.
26. NEEF, Sebastian. *Hacking with LaTeX* [online]. 2016. [cit. 2026-03-21]. Dostupné z: <https://web.archive.org/web/20260209043241/https://0day.work/hacking-with-latex/>.
27. SMARTBEAR SOFTWARE. *Swagger UI* [online]. 2026. [cit. 2026-03-24]. Dostupné z: <https://swagger.io/tools/swagger-ui/>.

Obsah příloh

	readme.txt	stručný popis obsahu média
	src	
	akrmata_backend	zdrojové kódy implementace backendu
	akrmata_frontend	zdrojové kódy implementace frontendu
	akrmata_latex	zdrojové kódy implementace LaTeX mikroslužby
	form_examples	ukázky standardizovaných formulářů
	thesis	zdrojová forma práce ve formátu LaTeX
	text	text práce
	thesis.pdf	text práce ve formátu PDF